



廈門大學
XIAMEN UNIVERSITY

游戏设计 & 设计思维

线下研讨课-6

Tutorial-6

主讲：林俊聪

邮箱：jclin@xmu.edu.cn





contents

目录

- 声音
- 网络



廈門大學
XIAMEN UNIVERSITY

1 声 音

--自强不息，止于至善--



廈門大學
XIAMEN UNIVERSITY

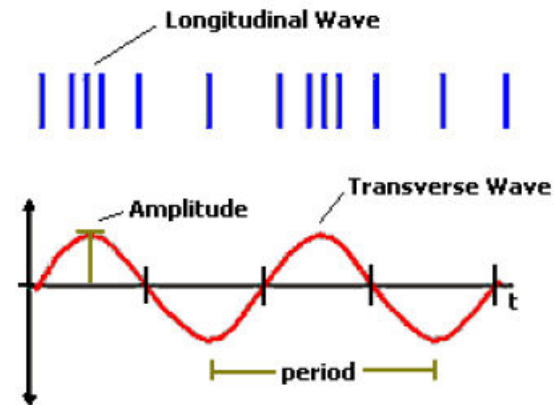
1.1 背景

--自强不息，止于至善--

The Nature of Sound

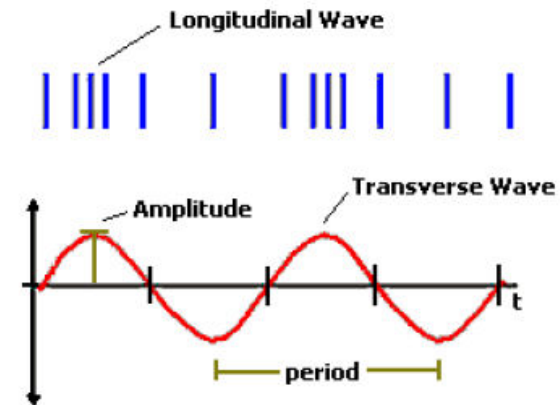


- What is Sound?:
 - **Vibratory energy** caused by movement of **physical objects**
 - As mechanical wave energy, it requires a **medium** such as air or water in which to propagate
- Properties:
 - Frequency: **Rate** of vibration
 - Experience as **pitch** (high or low)
 - Amplitude: **Intensity** of vibration.
 - Experience as **loudness**
 - Measured in **decibels** (dB) (too loud/too long hearing loss)

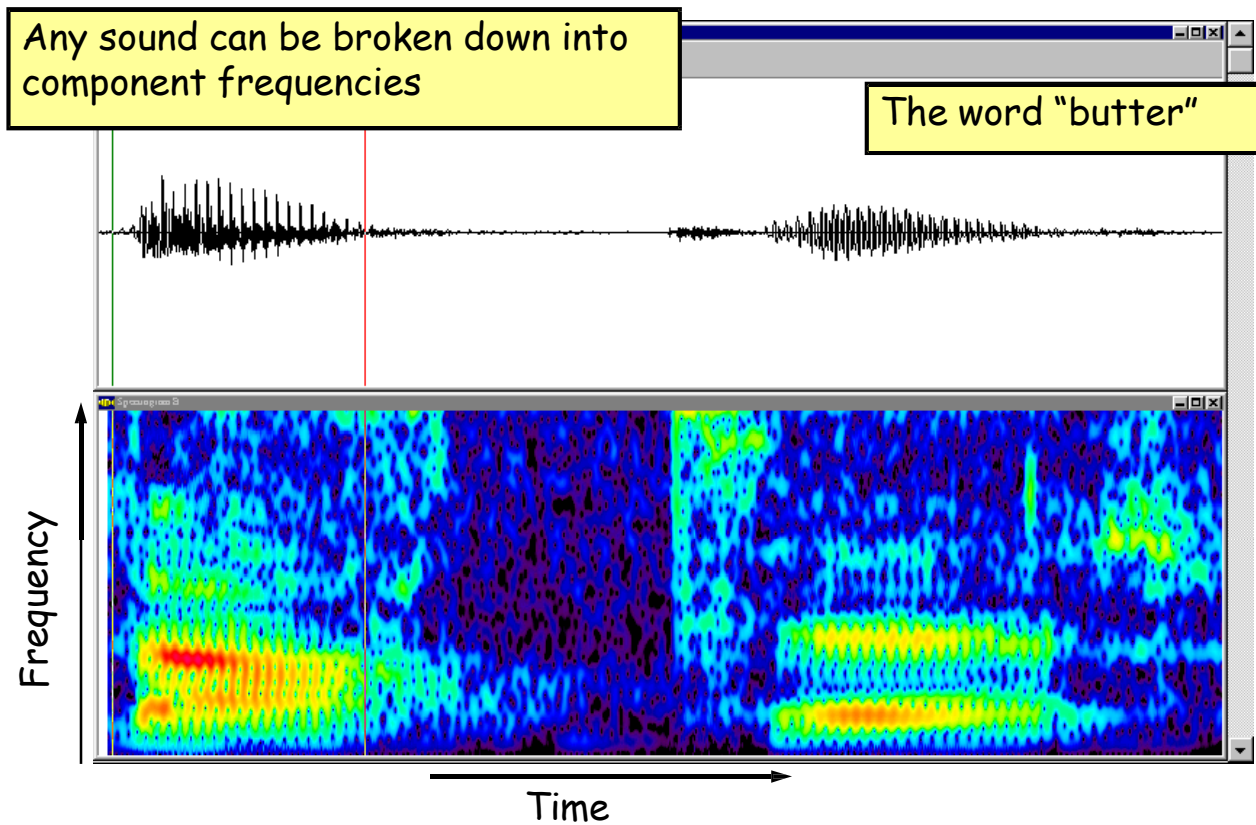


The Nature of Sound

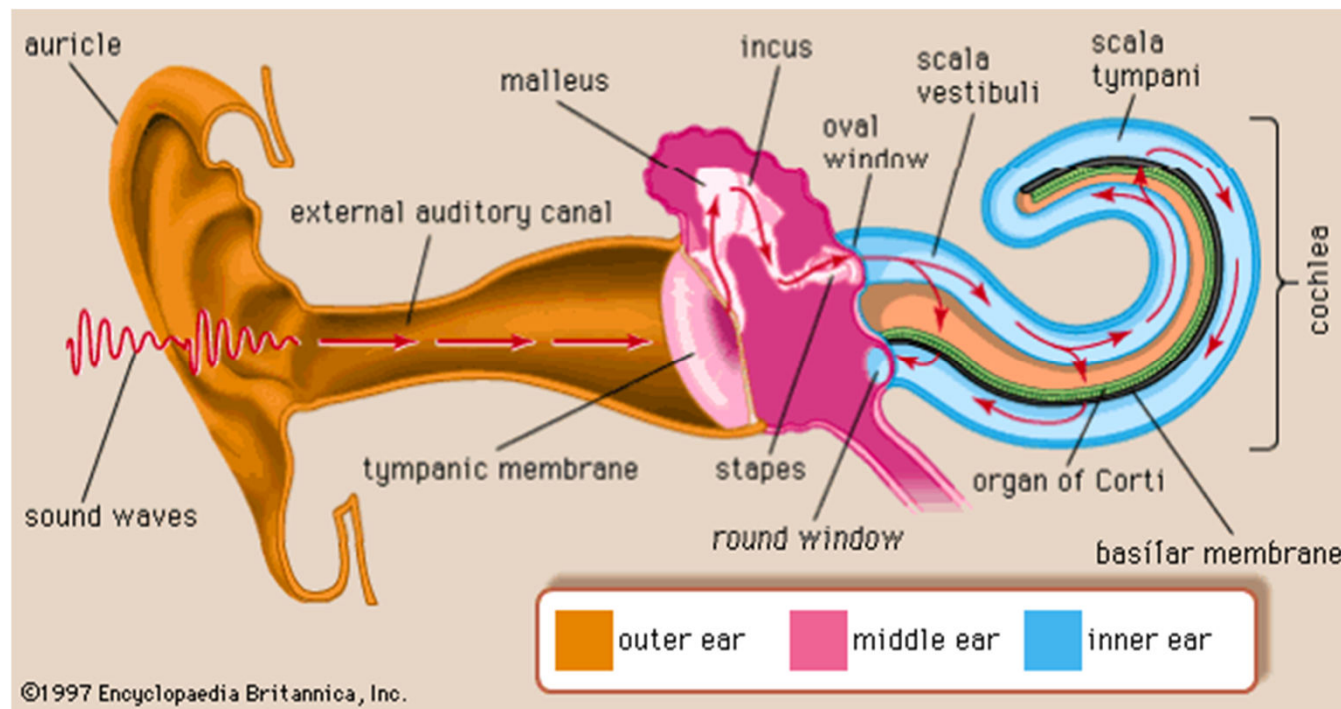
- Frequency:
 - Number of sound pulses that travel past a fixed point in a second
 - Measured in Hertz (Hz), that is, cycles per second
 - Humans: 20-20,000 Hz; Bats: 100,000 Hz
- Speed of Sound:
 - How fast sound pulse travels
 - Sound travels at same speed in a given medium, irrespective of pitch
 - – 344 meters/sec in air
- Pitch:
 - As with color in vision, this is perception
 - A high frequency sound is heard as a high pitch.



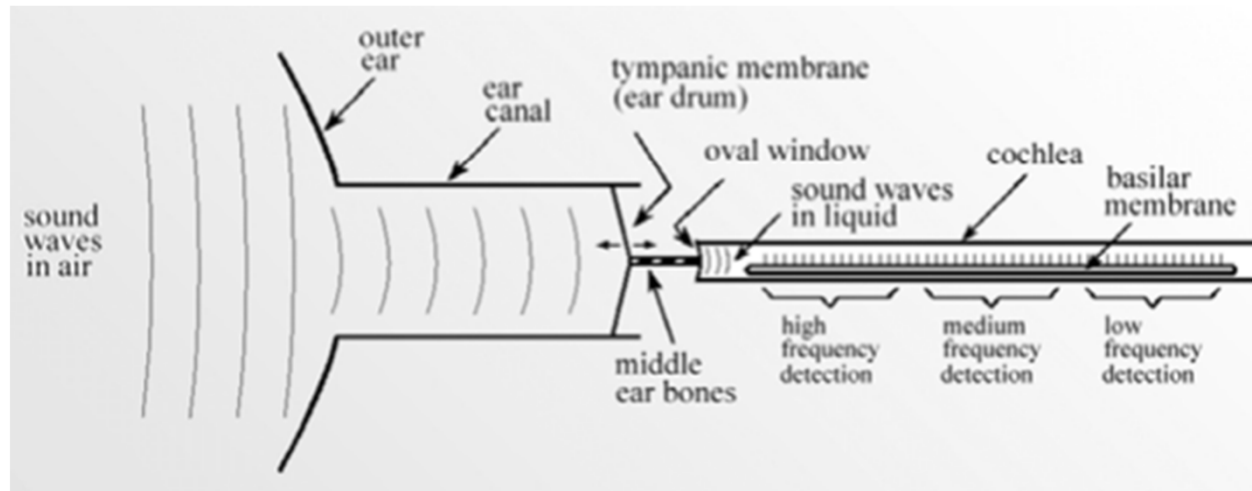
The Nature of Sound



Human Hearing



Human Hearing



- Outer ear collects sound waves from the environment
- Ear drum → middle ear bones → fluid-filled cochlea
- 12,000 nerve cells form the cochlear nerve
- Each nerve cell only responds to narrow range of audio frequencies
- The ear is a frequency spectrum analyzer

Volume



- Perception of volume **logarithmic** in **amplitude**
- A **decibel** is proportional to the log of the ratio of two intensities:
 $10 \cdot \log_{10}(I_1/I_2)$
- A 3 decibel (dB) increase is roughly equivalent to amplitude **doubling**
- As **absolute measure**, it's in comparison to **threshold of audibility**
 - 0 dB: **inaudible**
 - 60 dB: normal speech
 - 80 dB: a **shout**, city street **traffic noise**
 - 100 dB: motorcycle/snowmobile
 - 120-130 dB: front row at a **heavy metal rock** concert
(AC/DC, Led Zeppelin, Deep Purple)
 - 140 dB: **gun blast**

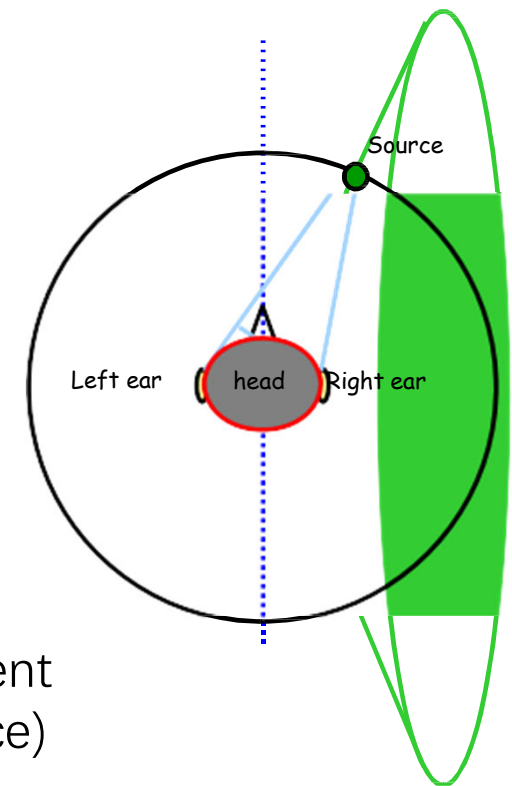
Pitch



- Perception of pitch is **logarithmic** in **frequency**
- **Higher frequencies** are perceived as higher pitches
- Measured in **hertz** Hz (cycles per second) and **kilohertz** kHz.
- **Humans audible range**: roughly 5 to 20,000 Hz
- **Examples**:
 - 8 Hz: Lowest frequency produced by **whales**
 - 32 Hz: Lowest note on **piano**
 - 250 Hz: Typical **speaking voice**
 - 440 Hz: Baby's **cry**
 - 4096 Hz: Highest note on a **piano**
 - 200,000 Hz: Highest frequency produced by **dolphins**

How do we perceive sound location?

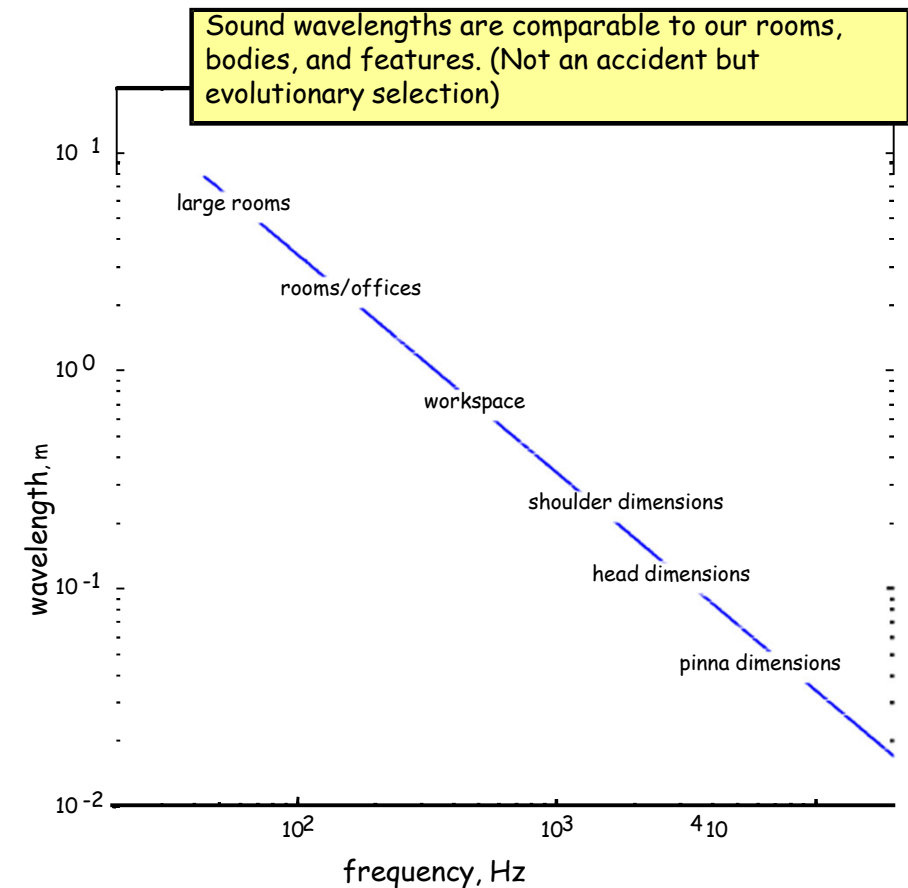
- Initial idea:
 - Measure attributes of received sound at the two ears
- Compare sound received at two ears:
 - Interaural **Level** Differences (ILD)
 - Interaural **Time** Differences (ITD)
- Surfaces of constant Time Delay:
 - $|x - x_L| - |x - x_R| = c \cdot \delta \cdot t$
 - Delays same for points on **cone-of-confusion**
 - Level differences are **small**
- Other mechanisms necessary:
 - **Scattering** of sound off our **bodies** and off the environment
 - **Purposive motion** (e.g. moving left-right to localize source)



Sound and Human Spaces

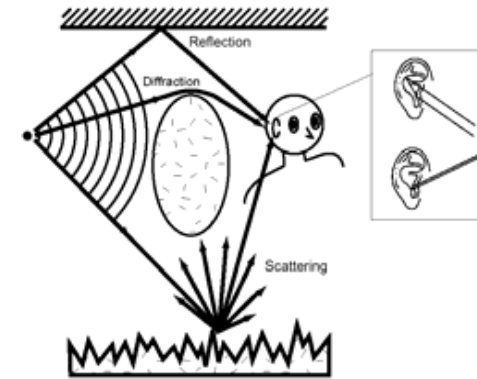


- Sound Wavelengths:
 - Comparable to human dimensions and of the spaces we live in
 - Wavelength λ is inversely proportional to frequency f
- Cases: a = object size
 - $\lambda \gg a$: the wave is not affected by object
 - $\lambda \sim a$: scattered wave is complex and diffraction effects are important
 - $\lambda \ll a$: wave behaves like a ray



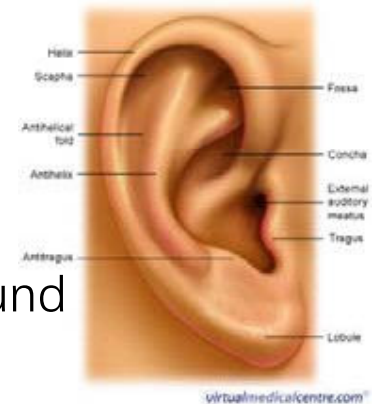
Modeling Sound Scattering

- Interactions change received sound waves.
- Scattering of body and ears:
 - Bodies ~ 50 cm
 - Heads ~ 25 cm
 - Ears ~ 4 cm
 - Not much multiple scattering
- Scattering off surroundings:
 - Rooms ~ 2m – 10m
 - More multiple scattering
 - Larger sizes lower frequencies



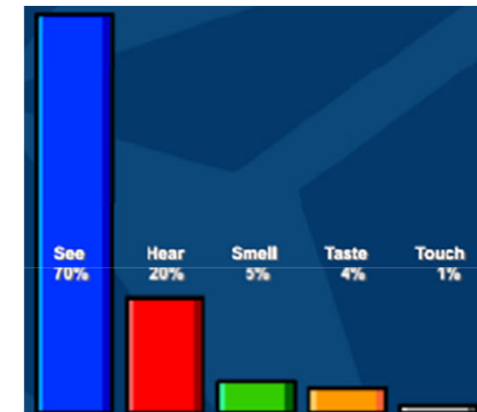
Pinna (Outer Ear) Shape and Hearing

- Pinna (or Auricle):
 - The **outer ear** with its complex **contours**
- Low frequency sounds:
 - Pinna behaves similarly to a reflector dish, thus amplifying the sound
- High frequency sounds and Pinna Notch:
 - Some sounds enter ear directly while others reflect off the contours of the pinna
 - Reflections enter the ear canal at a very slight delay
 - This delay results in phase cancellation, where the frequency component whose wave period is twice the delay period is virtually eliminated, called pinna notch
 - This allows us to perceive subtle variations in the locations of sounds
 - Each ear is different, so the effect varies for each individual



Sound in Games

- Role of Sound in Games:
 - “[Sound] serves the **story**, creates a **mood**, ... and can be the key to bringing **the visuals to life**” [LoBrutto,1994]
 - **20% of perception is acoustic** [Dobbler et. al. 2002]
- Sound Effects:
 - Indicates **what is going** on around the player
 - May alert player to **significant events**
 - May include **background sounds** and **musical score**
- Narration:
 - **Reinforces action**
 - Explains **non-obvious events** and story



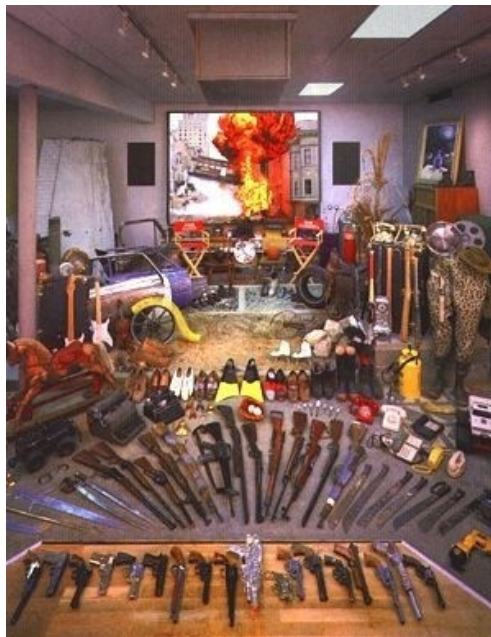
Sound Effects



- Background (or ambient) sound effects:
- Do **not** explicitly synchronize with the picture— **indicate setting**
- **Examples:** Wind, rain, traffic, crowd noises (called “**walla**”)
- Hard sound effects:
- **Common sounds** that appear **on screen**
- **Examples:** Door slams, weapons firing, and cars driving by
- Foley sound effects:
- Sounds that **synchronize precisely** with action on screen
- Require the **expertise of a Foley artist** to record properly
- **Examples:** Footsteps punching rustling of cloth
- Footsteps, punching, Design sound effects and background score:
- Sounds that **do not occur in nature** or impossible to record
- Commonly used **in science fiction and fantasy** genres
- Use of **music** to create an emotional mood

How can it be done?

- Foley artists manually make and record the sound from the real-world interaction



Lucasfilm Foley Artist

Foley Artists

- Ever wondered?
 - Why footsteps sound louder in movies?
 - Why they echo so much?
 - Why the clanging of swords is so ringing and metallic?
- Answer:
 - **Foley artists.**
 - Named after Jack Foley, a famous 1930's sound engineer.
- Web sites:
 - <http://www.marblehead.net/foley/index.html>
 - http://www2.digidesign.com/digizine/-dz_main.cfm?edition_id=142&navid=1053



Foley Artists

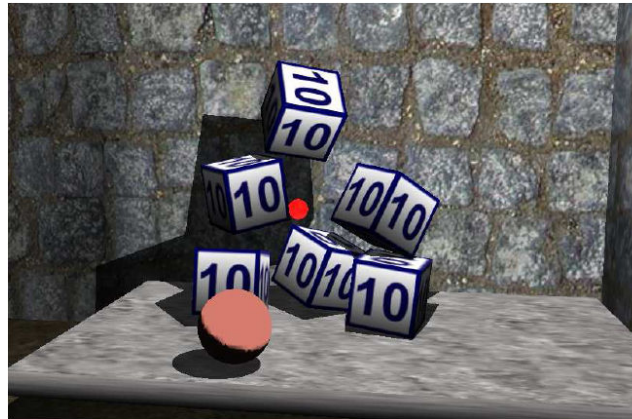


- Effect:
 - Galloping horses
 - Kissing
 - Punching someone
 - High heels
 - Bone-breaking blow
 - Footsteps in snow
 - Star Trek sliding doors
 - Star Wars sliding doors
 - Bird flapping its wings
 - Grass or leaves crunching
- How it might be made:
 - Banging empty coconut shells together
 - Kissing back of hand
 - Thumping watermelons
 - High heels on wooden platform
 - Breaking celery or bamboo or twisting a head of lettuce
 - Squeezing a box of corn starch
 - Pulling a piece of paper from an envelope
 - Flare gun plus sneakers squeak
 - Flapping a pair of gloves
 - Balling up audio tape

How about Computer Simulation?

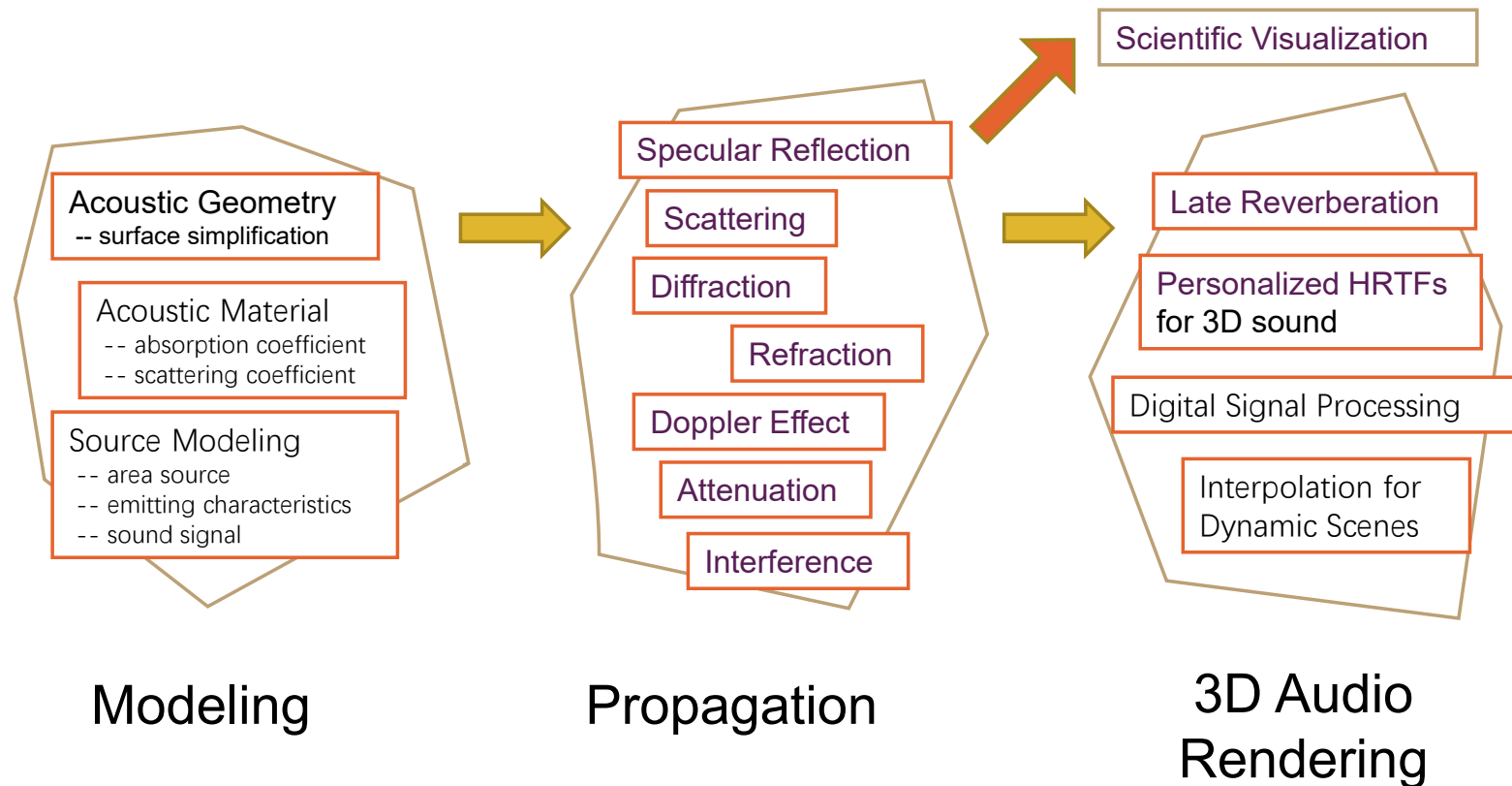


- Physical simulation drives visual simulation
 - Sound synthesis can also be automatically generated through physical simulation



Videos w/ and w/o audio

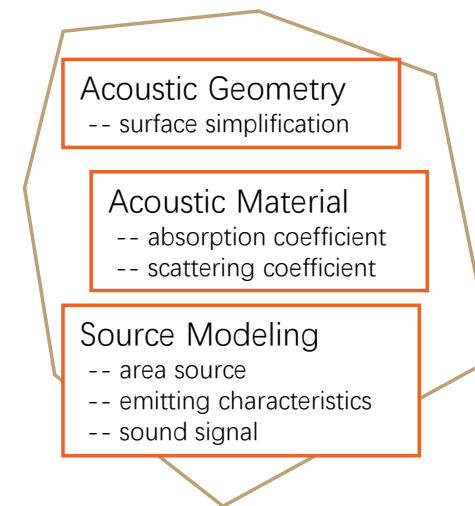
Sound Rendering: An Overview



Sound Rendering: An Overview



- Modeling
 - Acoustic vs. Graphics
 - Low geometric detail vs. High geometric detail

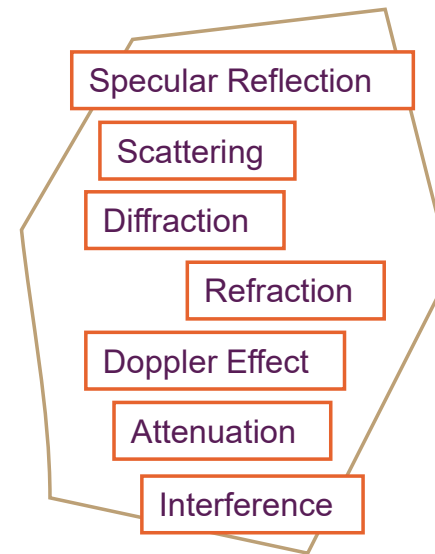


Modeling

Sound Rendering: An Overview



- Propagation
 - Acoustic vs. Graphics
 - 343 m/s vs. 300,000,000 m/s
 - 20 to 20K Hz vs. RGB
 - 17m to 17cm vs. 700 to 400 nm

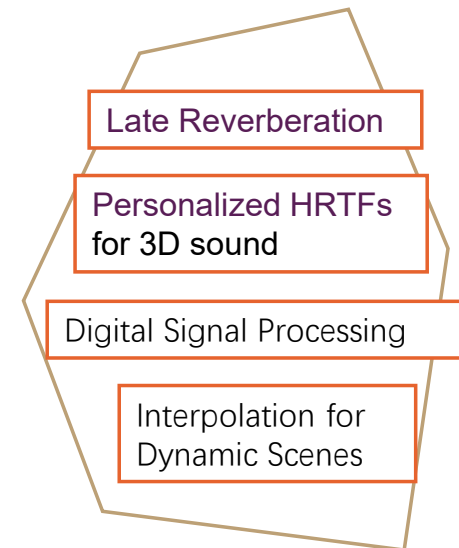


Propagation

Sound Rendering: An Overview



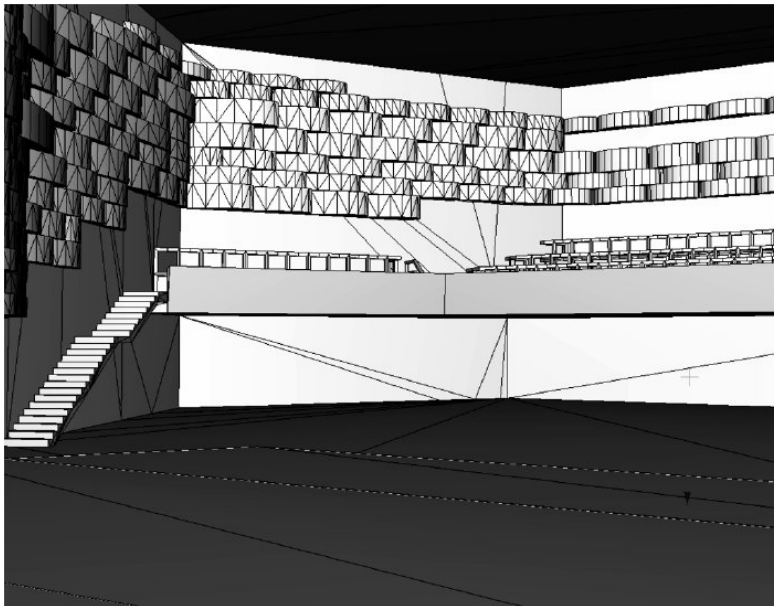
- 3D Audio Rendering
 - Acoustic vs. Graphics
 - Compute intensive DSP vs. addition of colors
 - 44.1 KHz vs. 30 Hz
 - Psychoacoustics vs. Visual psychophysics



3D Audio
Rendering

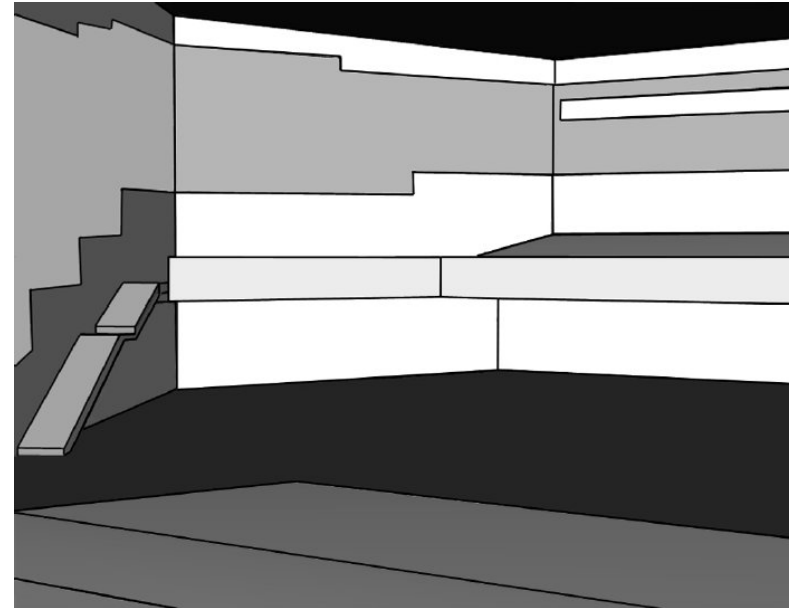
Modeling

- Modeling Acoustic Geometry



Visual Geometry

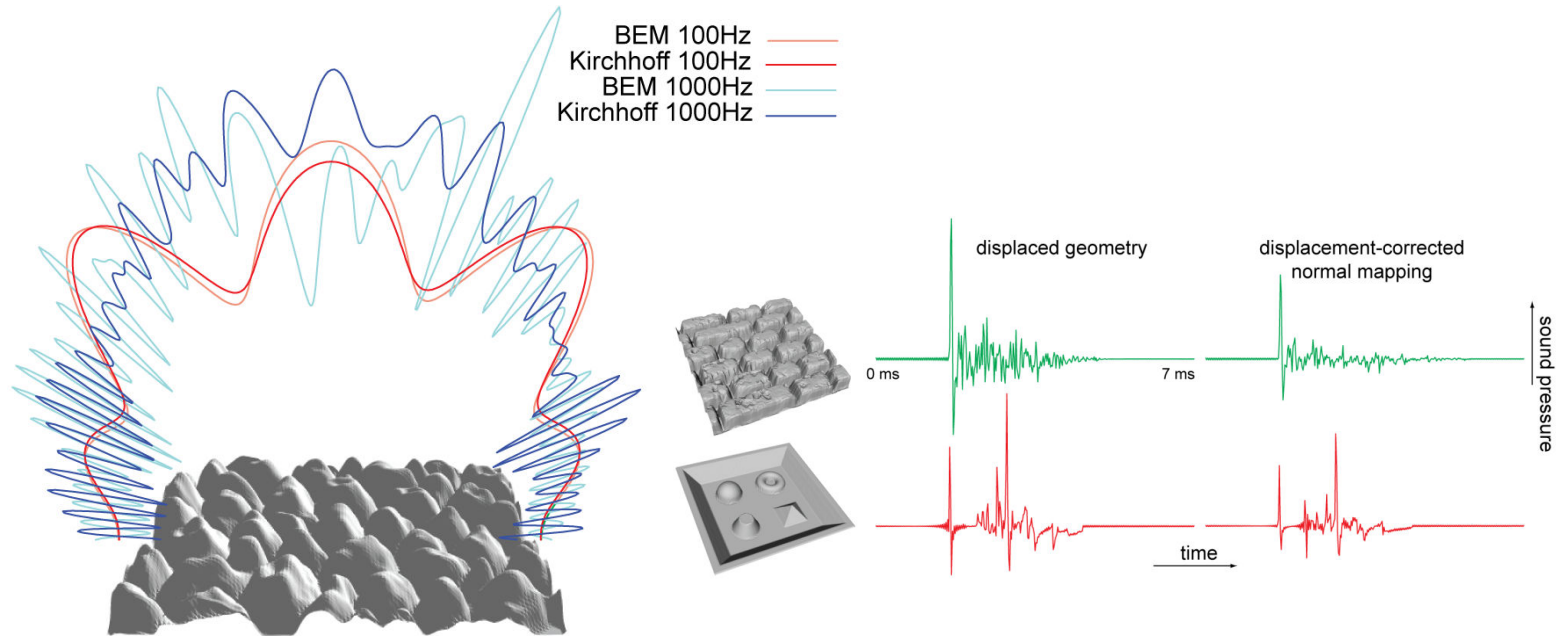
[Vorländer,2007]



Acoustic Geometry

Modeling

- Modeling Sound Material



[Embrechts,2001] [Christensen,2005] [Tsingos,2007]

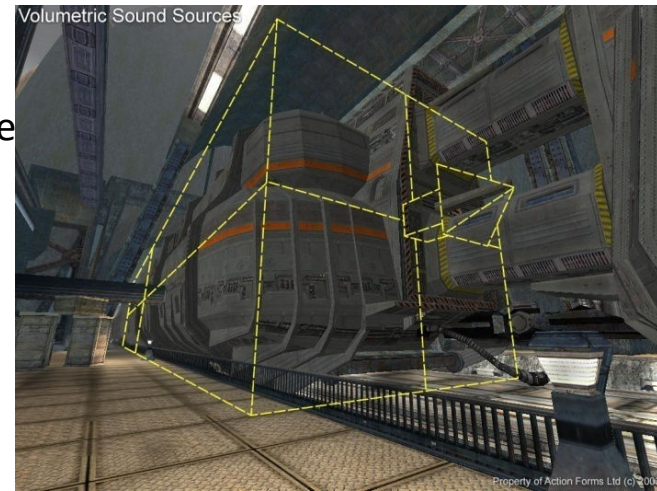
Modeling

- Modeling Sound Source

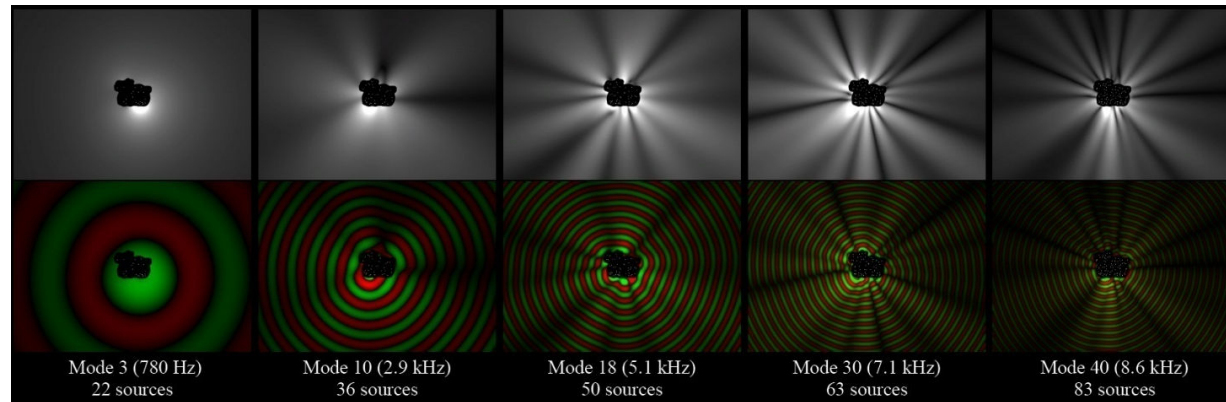


Directional Sound Source

Volumetric Sound Source



Complex Vibration Source

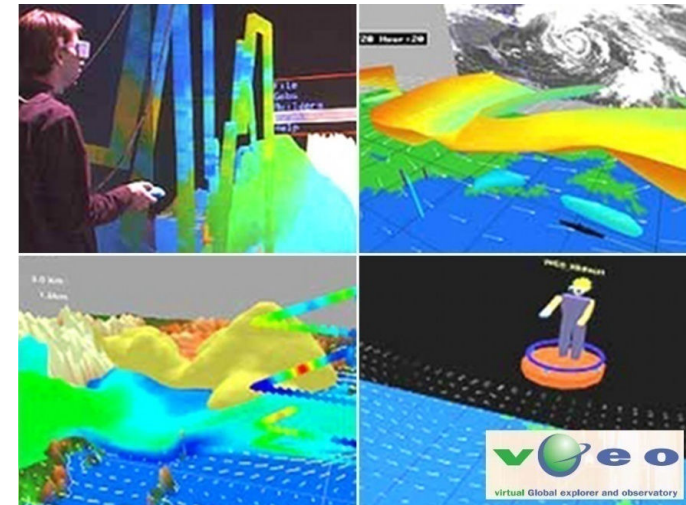


Propagation

- Applications
 - Advanced Interfaces
 - Multi-sensory Visualization



Minority Report (2002)



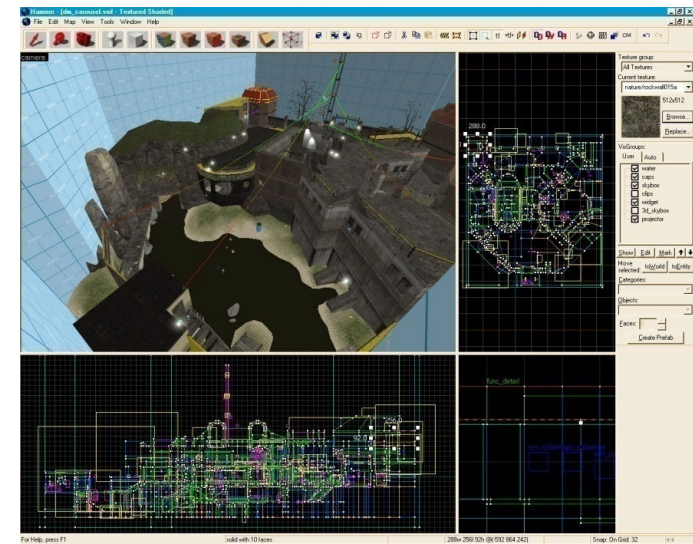
Multi-variate Data Visualization

Propagation

- Applications
 - Acoustic Prototyping



Symphony Hall, Boston



Level Editor, Half Life

Propagation

- Applications
 - Games
 - VR Training



Game (Half-Life 2)



Medical Personnel Training

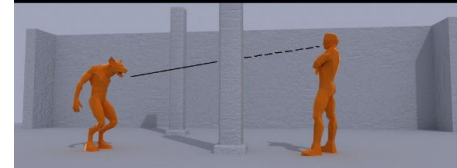
Propagation

- Sound Propagation in Games
 - Strict time budget for audio simulations
 - Games are dynamic
 - Moving sound sources
 - Moving listeners
 - Moving scene geometry
 - Trade-off speed with the accuracy of the simulation
 - Static environment effects (assigned to regions in the scene)



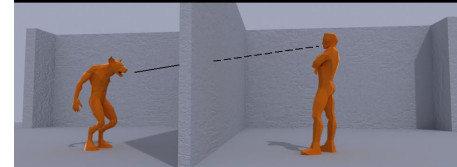
廈門大學
XIAMEN UNIVERSITY

Obstructions



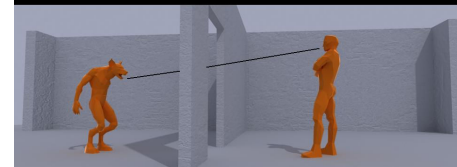
- Direct path is muffled
- Reflections are clear

Occlusions



- Direct path is muffled
- Reflections are muffled

Exclusions



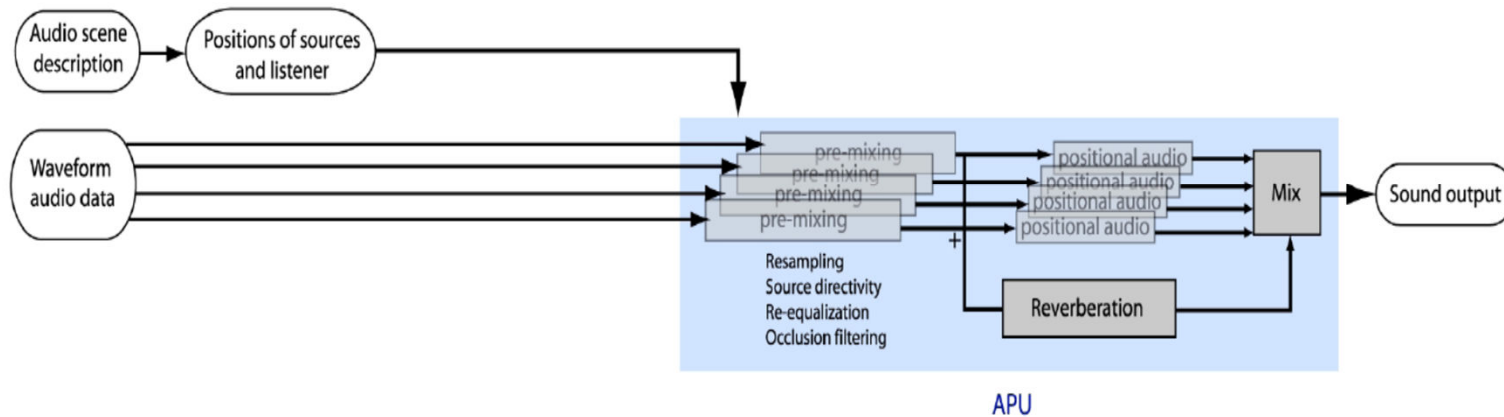
- Direct path is clear
- Reflections are muffled

3D Audio Rendering

- Main Components
 - 3D Audio and HRTF
 - Artifact free rendering for dynamic scenes
 - Handling many sound sources

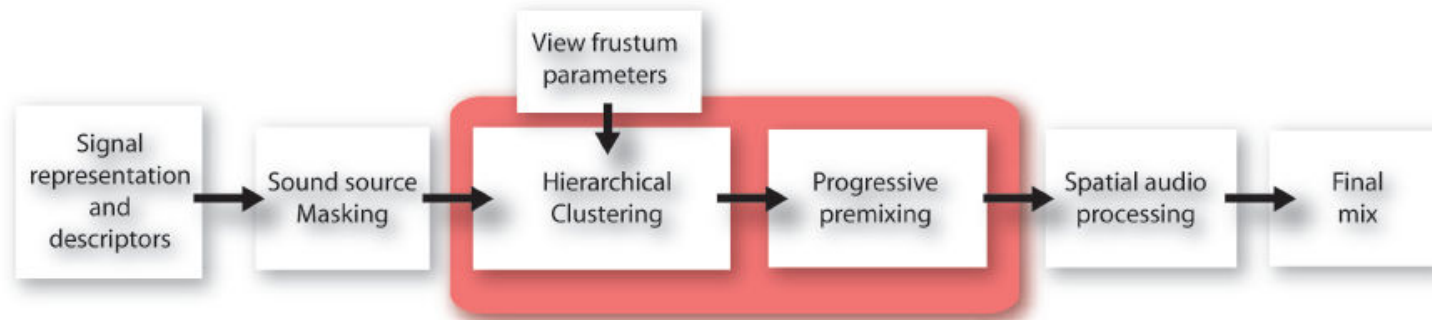
Need to cite the authors of these images.

Traditional pipeline

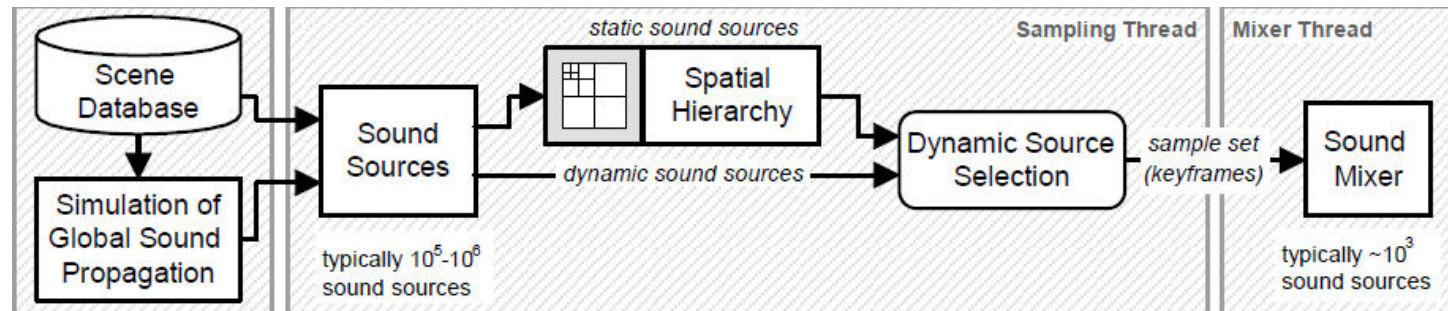


3D Audio Rendering

- Perceptual Audio Rendering [Moeck,2007]



- Multi-Resolution Sound Rendering [Wand,2004]





廈門大學
XIAMEN UNIVERSITY

1.2 Sound in VR

--自强不息，止于至善--



VR and sound



- Sense of presence and immersion
- Situational awareness
- Additional information about environment
- Better audio-visual correlation

What about VR makes sound different?

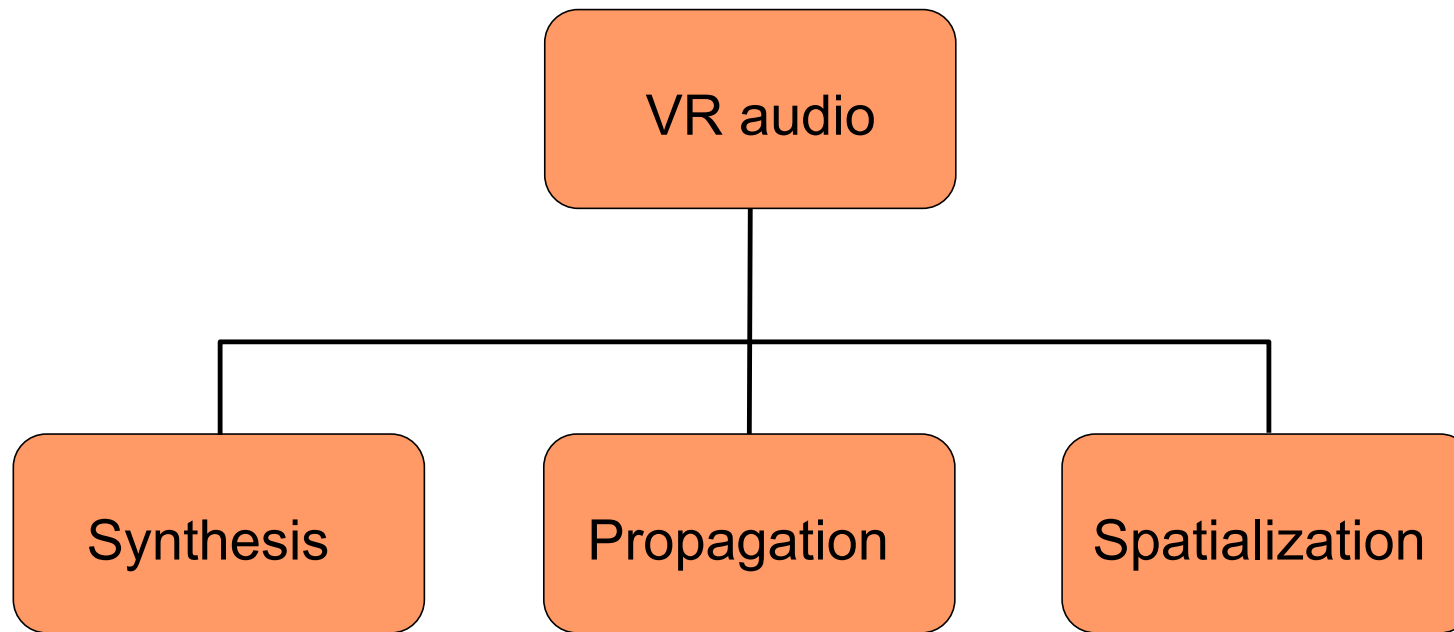


- You are part of 3D virtual world rather than watching on 2D screen
- Fidelity and accuracy requirement significantly higher
- Interact with the virtual environment

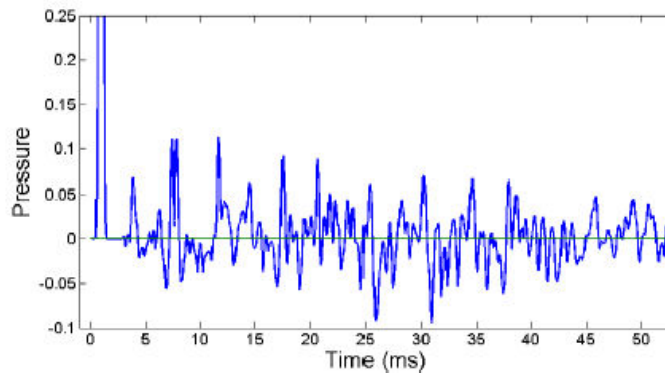
Physically-based Sound Simulation



廈門大學
XIAMEN UNIVERSITY



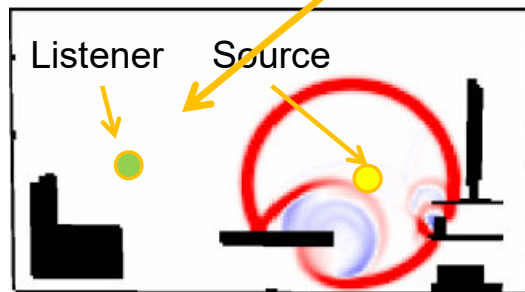
Sound propagation



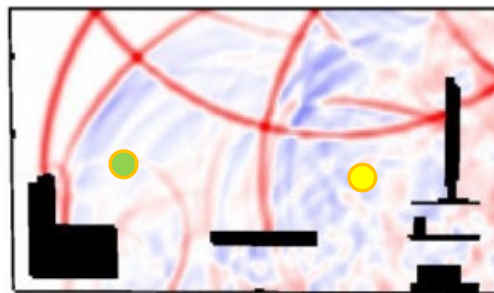
Propagation filter

■ High pressure

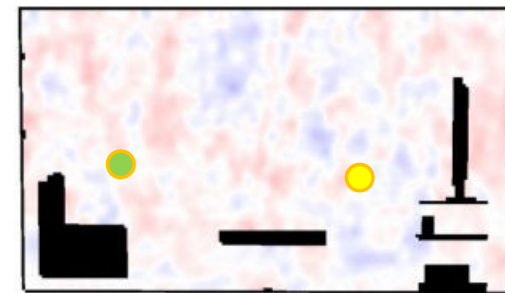
■ Low pressure



$t = 2$ ms



$t = 8$ ms



$t = 50$ ms

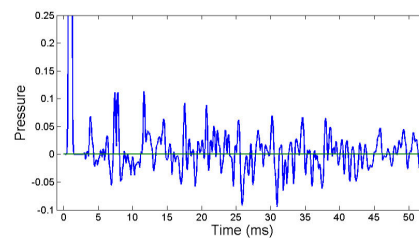
Sound propagation filter

dry audio



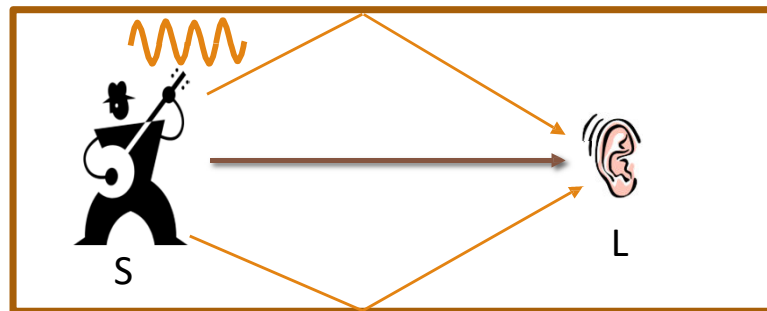
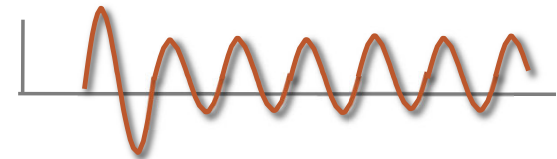
*

propagation filter



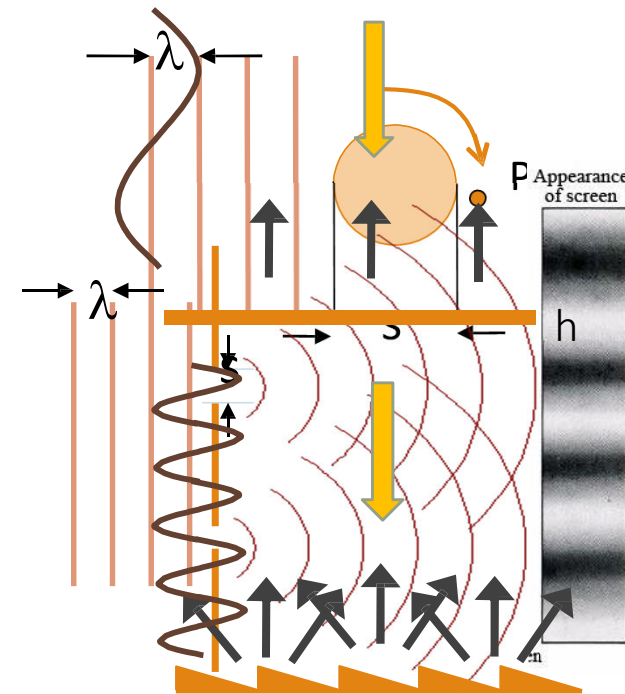
=

propagated audio



Sound propagation effects

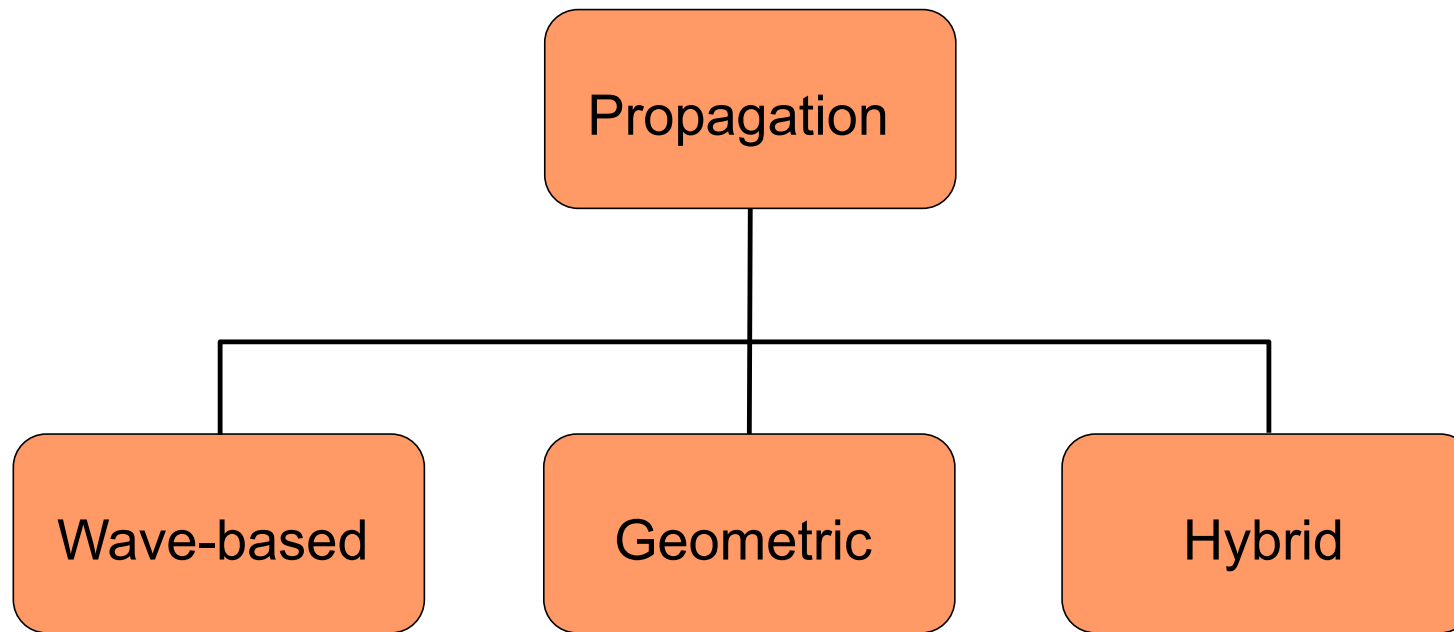
- wavelength $\sim$ object size
 - diffraction
 - interference
- wavelength $\ll$ object size
 - specular reflection
 - scattering



Physically-based Sound Simulation



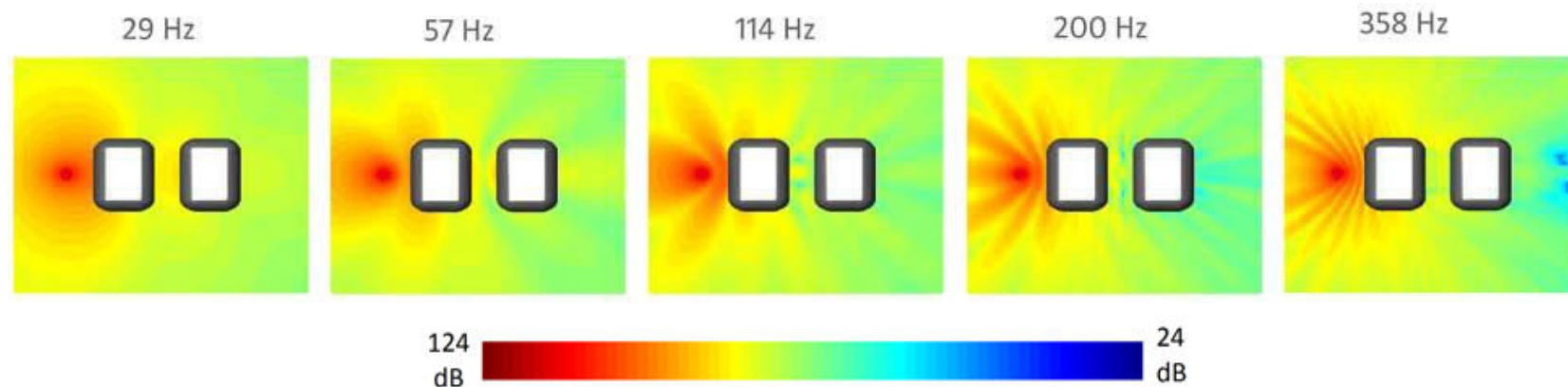
廈門大學
XIAMEN UNIVERSITY



Wave-based techniques

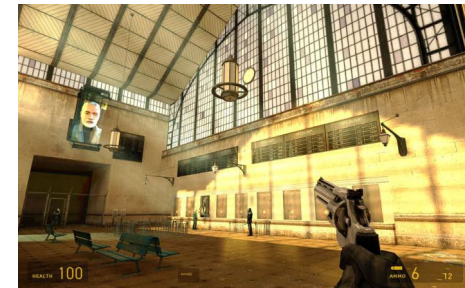
- Solves underlying physics of sound propagation

$$\nabla^2 P + \frac{\omega^2}{c^2} P = 0$$



Wave-based techniques

- Sound radiation
 - precomputed acoustic transfer [James 2006]
- Sound Propagation
 - medium scenes, dynamic sources [Raghuvanshi 2009]
 - large scenes, directivity [Mehra 2014]
 - + dynamic sources [Mehra 2015]



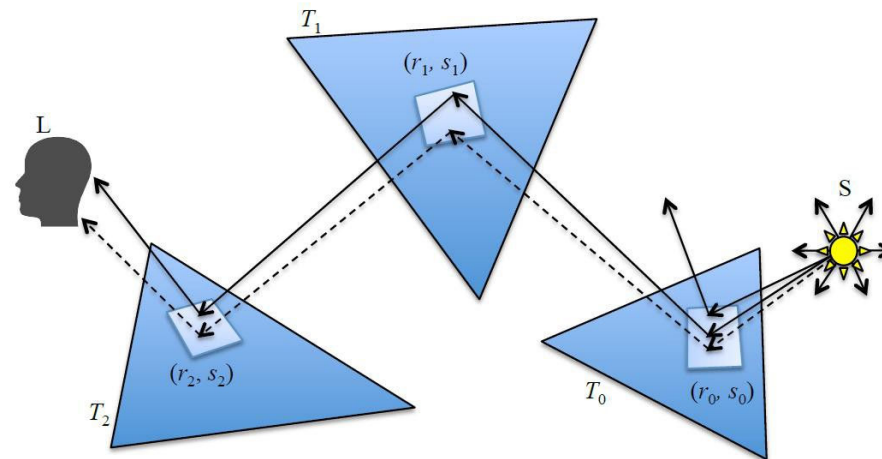
Wave-based techniques



- Accurate at all frequencies
 - reflection, diffraction, interference
- Precomputation $\propto \text{freq}^4$
 - hours on CPU cluster (low freq.), days to months (high freq.)
- Interactive runtime
 - 10s of MBs, < 50 ms update rate

Geometric techniques

- Ray-like behavior of sound waves
 - valid for high-frequencies
 - similar to global illumination
 - ray-tracing, visibility graph, etc.



Geometric techniques

- Typical techniques
 - ray-tracing [Krokstad 1968], image sources [Allen 1979]
 - beam tracing [Funkhouser 1998], phonon tracing [Bertram 2005]
- Approximate diffraction
 - uniform theory of diffraction (UTD)
- Combined techniques
 - ray-tracing, UTD



Geometric techniques



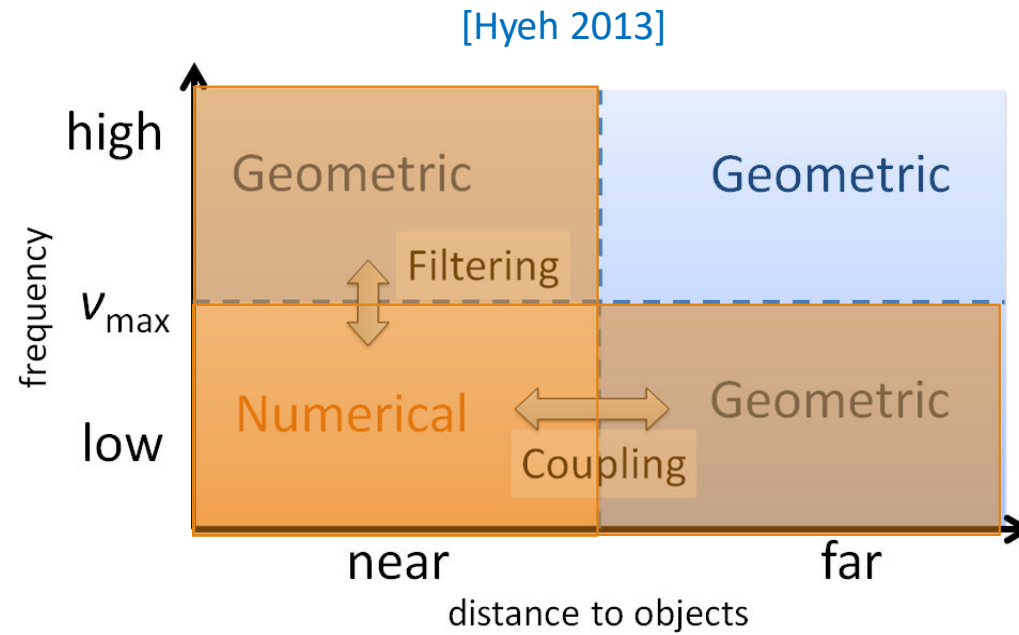
- Approximate at low, accurate for high frequencies
 - reflection, diffuse reflection
- Negligible precomputation
 - mins on single desktop
- Interactive runtime
 - 10s of MB, < 100 ms update rate

Hybrid techniques



- Wave-based techniques
 - efficient at low freqs., expensive at high freqs.
- Geometric
 - inaccurate at low freqs., accurate at high freqs.
- Hybrid = wave + geometric
 - accurate, efficient for all freqs.

Hybrid techniques



Challenges



- VR game audio
 - source, listener position, scene geometry
 - compute propagation filter dynamically
- Access to dynamic scene geometry

Precomputation budget



- Sound propagation is computationally expensive
 - multiple freqs., wave-effects
- Analogous to light propagation
 - CPU cluster for light maps
 - tens of hours of cluster time

Runtime resources



- Audio processing
 - < 10% of single CPU, < MBs of memory
- Graphics processing
 - dedicated graphics processor
 - GBs of memory
- Runtime budget
 - at least 1 core, tens to hundreds of MBs

Conclusion



- Accurate sound simulation is important for VR
- Physically-based propagation techniques
 - Wave-based, geometric and hybrid
- Unsolved problems
 - dynamic geometry, pre-computation cost, real-time runtime



廈門大學
XIAMEN UNIVERSITY

1.3

Integration in to Game Engines

--自强不息，止于至善--

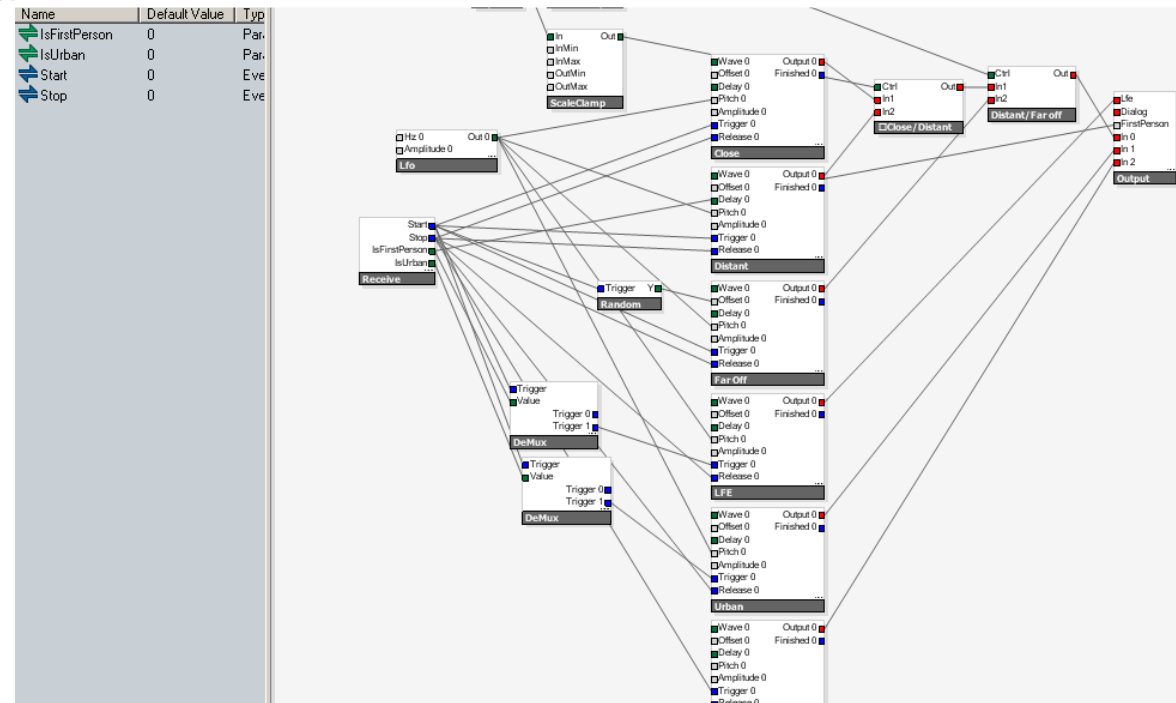
Components of a game audio engine



廈門大學
XIAMEN UNIVERSITY

- Codecs
- Synthesis / Audio « shaders »

A gun sound “shader” in
Battlefield Bad Company © EA DICE

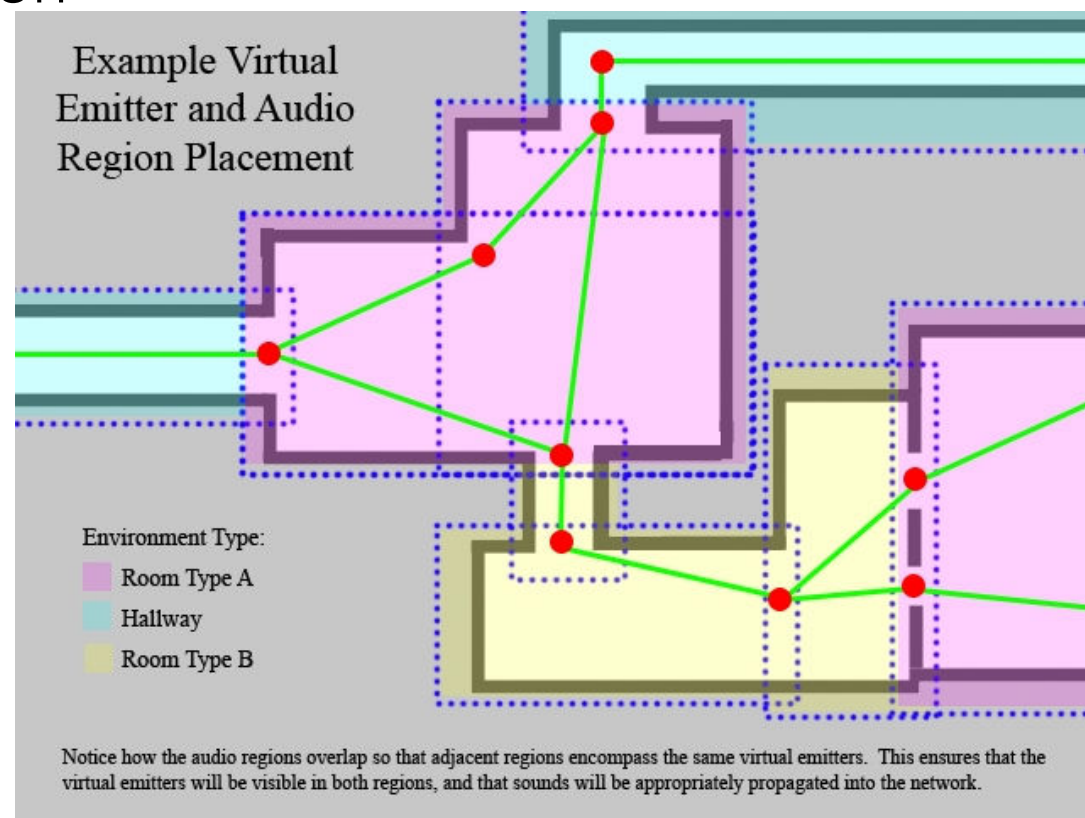


自强不息，止于至善

Components of a game audio engine

- Environmental sound propagation
- DSP effects

Virtual emitter technology for SOCOM confrontation
© SlantSix Games

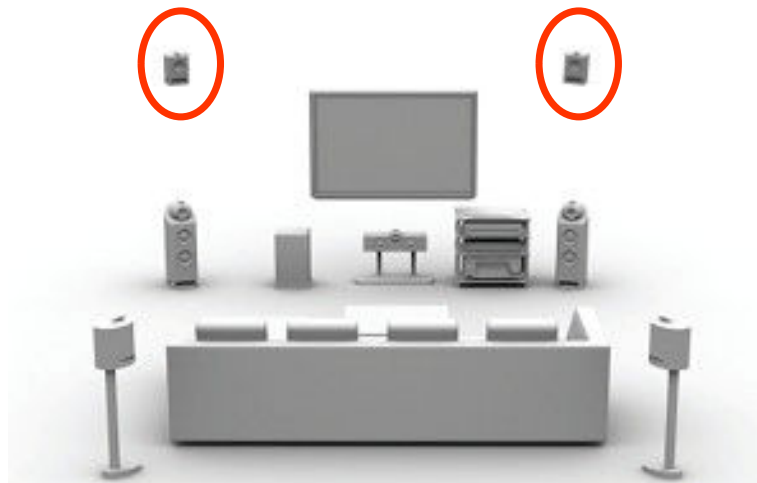


Components of a game audio engine



廈門大學
XIAMEN UNIVERSITY

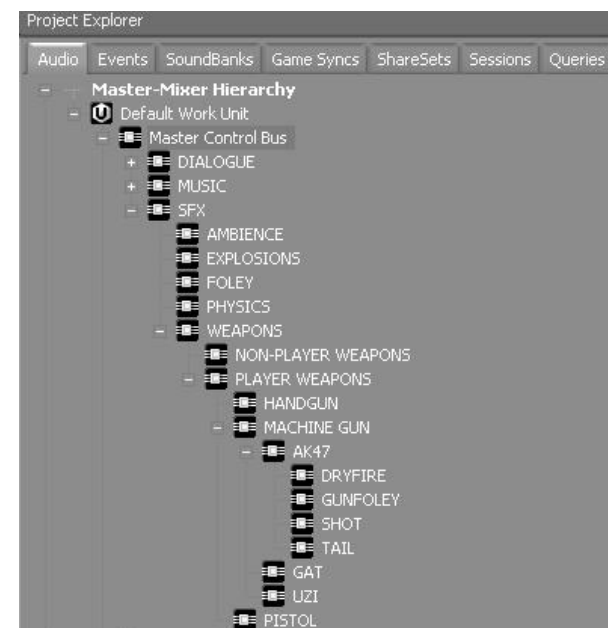
- 3D audio rendering



Dolby Prologic IIz configuration
with height channels

Components of a game audio engine

- Complex bus routing and mixing
- Mastering, leveling and dynamic range control



Bus grouping in Wwise
© Rob Bridgett

Middleware or in-house ?



- Powerful middleware available
 - Wwise, FMOD, Niles, XAudio, SCREAM replace DirectSound/openAL/EAX
 - real-time processing engines and authoring tools
 - provide visual interfaces to sound designers
 - interactive scripting
 - manage large databases of assets
 - debugging and profiling tools
- State-of-the-art techs still tend to be implemented in-house
 - differentiating factor

Integrating advanced effects in game engines



廈門大學
XIAMEN UNIVERSITY

- Perceptual rendering
 - clustering and masking
- Frequency-domain pipeline
 - Time vs. frequency domain
 - Geometrical propagation

Clustering for game audio

- First use of perceptual audio rendering in commercial game engine
 - Next-gen platforms
 - Test Drive Unlimited and Alone in the Dark: NDI
- Test-Drive unlimited data:
 - 200 to 400 sources
 - cars and « physics » clusters
 - error-based refinement
 - 12 clusters sufficient
 - x3 speed-up for 5.1 rendering



Perceptual routing for VoIP

- In-game spatialized chat
 - Optimize bandwidth
 - Use a forwarding bridge
- Bridge performs on the fly masking
 - 70% audio frames discarded
 - Clients perform additional masking+clustering with other in-game sounds

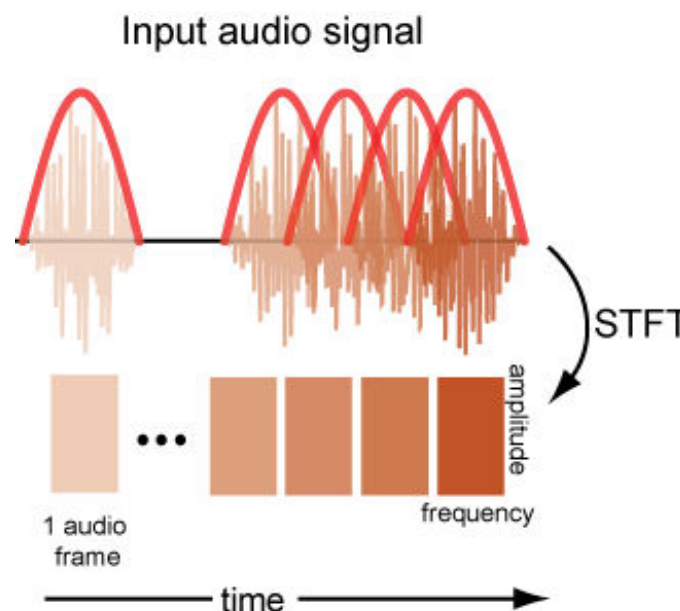


Time vs. frequency domain processing

- Most game audio pipelines perform time-domain processing
 - IIR filtering, mixing, pitch-shifting
 - Fourier-domain FIR convolution
- Advantages of time-domain processing
 - sample-accurate
 - pitch-shifting
 - no transforms
- Advantages of frequency domain
 - arbitrary filtering, subband processing is possible (and fast)
 - progressive/level-of-detail rendering
 - integrates with the codec !

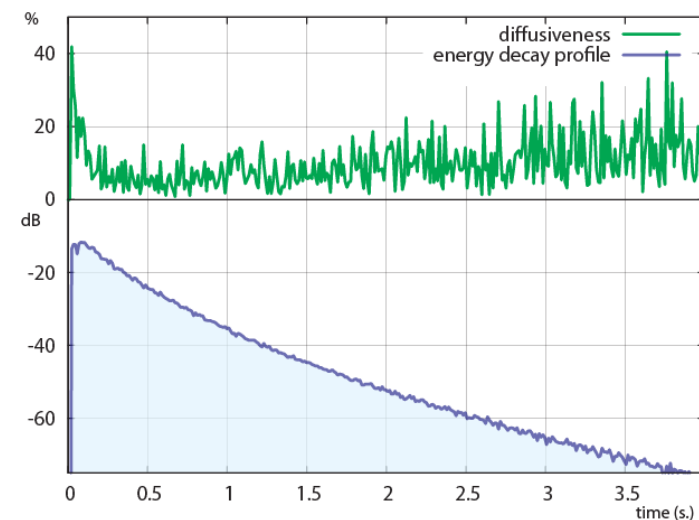
Implementing a frequency domain rendering pipeline

- Choose your transform
 - FFT vs. DCT
 - can be built-in your codec !
- Processing and mixing
 - EQ/convolution/granulation
 - pitch shifting is harder !
 - Zero-padding might not be necessary
- Overlap-add reconstruction
 - apply windows twice before and after the transform (e.g., sine)
 - one inverse transform per output channel is OK



Frequency domain pipeline for geometrical acoustics

- Reflections and late reverberation
- Early reflections as image-sources
- Late reverberation as parametric model
 - pre-computed
 - hybrid directional/diffuse rendering model



What next ?



- Meta-data handling
 - computational decision making and performance
 - improve mix quality and creativity
- Integrated decoding, processing and mixing
 - perceptual processing
 - multi-core architectures
- Parametric spatial audio coding [DirAC, SAOC, MPEG surround]
 - Authoring tools ? Recording techniques ?
- Massively multiplayer games
 - audio rendering for thousands of clients (e.g., Dolby Axon)



廈門大學
XIAMEN UNIVERSITY

2 网 络

--自强不息，止于至善--



廈門大學
XIAMEN UNIVERSITY

2.1

Multiplayer Games and Networking

--自强不息，止于至善--

Multiplayer Games



- Multiplayer Games: A number of players communicating through a network
 - Persistent Games: Such as “World of Warcraft”, where state is maintained, regardless whether anyone is playing
 - Transient Games: Only exist while people are playing, and reset each time the server-side is reset
- Performance Issues:
 - Latency: How much time delay until global state to be updated?
 - Reliability: How often is data lost or corrupted?
 - Bandwidth: What is the rate of data transfer?
 - Security: How is the game-play protected from tampering/cheating?
- Tradeoffs:
 - All of these considerations interact, and trade-offs must be made



Multiplayer Factors: Event Timing

- Factors in Multiplayer Games:
 - Event Timing: How do player's **interact** with the game?
 - Shared Display: How do different players **perceive** the game state?
 - Connectivity: How are players **connected** and how do they **communicate/interact**?
- Event Timing:
 - Turn-based:
 - Player's move in **turns** (round robin). Other players wait
- Real-time:
 - Players move **simultaneously**
 - May need to handle **race-conditions** (e.g., two players attempt to acquire the same resource at the same time)
 - “**Twitch Games**” typically require very low latencies, less than 150 ms.

Multiplayer Factors: Shared Display



- Shared Display: for multiplayer games on a single platform
 - Full Screen:
 - Complete Player Visibility: Such as board games. Everyone sees everything at all times
 - Player Funneling: Restricts players to a small region of game space, moving with players
 - Turn-Based Screen Control: Active player controls the viewpoint
 - Split Screen: Each player has a separate portion of display through the use of separate viewports
 - Components: System maintains the following information for each player
 - Camera: Viewpoint
 - Cull Data: What portion of the environment is visible
 - Heads-up Display: Game stats displayed on top of scene
 - Map Data: Centered about the current player
 - Audio Effects: Need to be split among players as well
 - Issues: Maintaining consistency between views. Practical only for few players

Multiplayer Factors: Connectivity



廈門大學
XIAMEN UNIVERSITY

- Direct Link:
 - What: Connected directly typically through short connections
 - Examples: Serial and USB cable, wireless (infrared and Bluetooth)
 - Pro: Fast, reliable. Con: Few players, only small distances
- Circuit-Switched Network:
 - What: Unshared direct connection between endpoints
 - Example: Traditional public telephone system
 - Pro: Reliable, low-latency. Con: Low bandwidth, few players
- Packet-Switched Network:
 - What: Communication broken into small packets that are routed over a shared network
 - Example: Internet
 - Pro: Many players, large areas. Con: Variations in latency/bandwidth

Networking Basics



- Network:
 - A group of two or more computers connected together
 - Henceforth we consider **packet-switched networks**
 - Characteristics:
 - Scale: The area spanned by the network
 - **LAN**: (Local-area network) E.g., connecting one business or school
 - **WAN**: (Wide-area network) Connecting computers distributed over an entire city, state, country or the world
 - Topology: How the computers are **connected** together. Examples: Ring, star, bus, tree
 - Protocol: Agreed upon **rules for communicating** and routing information. Examples: TCP, IP, UDP, FTP, HTTP
 - Architecture: General **communication structure**. Examples: Peer-to-peer, client server

Networking Basics



- Protocol:
 - A **convention** for routing and transferring data over **packet switched networks**
 - Communication networks may be **unreliable** and may connect machines having **widely varying** manufacturers, operating systems, speed, data formats
 - Issues:
 - **Packet sizes**: Fixed or variable sized packets?
 - **Handshaking**: Communication exchange to ascertain how data will be transmitted (format, speed, etc.)
 - **Acknowledgements**: For receipt of data
 - **Error checking/correction**: Handling errors in data transmission
 - **Compression**: Reducing data size due to limited bandwidth
 - **Encryption**: To protect private data

Networking Basics



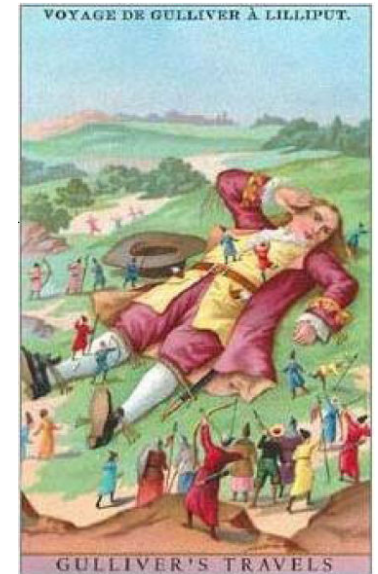
- Packet:
 - The logical transmission unit of a protocol
 - Two parts: Header (information) and payload (data)
 - Example: Quake network protocol packet structure:
 - <http://www.gamers.org/dEngine/quake/QDP/qnp.html>
 - Simple Example:

```
struct Packet {  
    // Header  
    short packetLength; // length of the packet (in bytes)  
    short packetType;   // E.g. data, control, acknowledgement  
    int checksum;       //checksum for error checking Payload  
    char data[256];     // ...the data  
};
```

Networking Basics

- Packet
 - Issues
 - Serialization:
 - Pointers and references cannot be reliably transmitted since they refer to local memory. Convert them to names or indices to an array
 - Abstract data types: Are often based on references for inheritance
 - Endianness: Transmitting multi-byte numbers: 0123
 - Little Endian: Low-order bytes first: 3, 2, 1, 0
 - Big Endian: High-order bytes first: 0, 1, 2, 3
- Intrinsic Types: Use `__int32` rather than `int` to
 - force compiler to use 32-bit integers
 - Unicode: Better than ASCII for string data

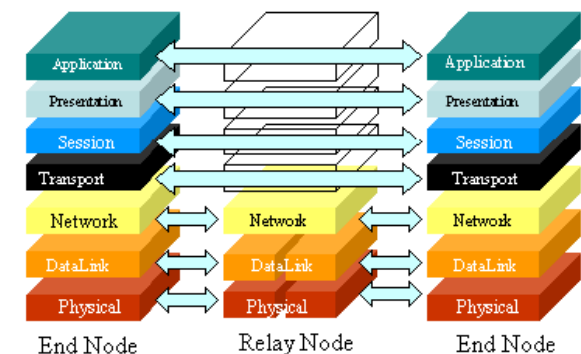
Irrelevant Trivia Alert: These terms come from the novel *Gulliver's Travels* by Jonathan Swift on a civil war between Lilliputian factions that cracked their hard-boiled eggs from the big end or the little end.



Networking Basics



- Open System Interconnect (OSI) Model: Formalizes the multilayered structure of networks
 - Application: End-user processes (e.g., mail (smtp), ftp, telnet)
 - Presentation: Packetization, byte order, encryption, compression
 - Session: Connection and data exchange (e.g., logging in/out, socket)
 - Transport: Flow control (e.g., TCP/UDP)
 - Network: Basic routing (e.g., IP)
 - Data Link: Packet/frame structure
 - Physical: Physical medium wire



Networking Basics



- Physical Layer
 - The medium over which data is carried
 - Examples: twisted-pair wire, coaxial cable, wireless
 - Latency and Bandwidth:
 - Time of flight: Time to send a single bit of data. Includes delays at switches from source to destination
 - Bandwidth: Maximum transfer rate from source to dest in bits per second (bps). Includes header
 - Transmission time: $\text{messageSize} / \text{bandwidth}$
 - Transport latency: $\text{timeOfFlight} + \text{transTime}$
 - Sender (receiver) overhead: Time to process packet for sending (receiving)
 - Total Latency: $\text{overheadsend} + \text{timeOfFlight} + \text{transTime} + \text{overheadrec}$
 - Effective Bandwidth: $\text{messageSize} / \text{totalLatency}$

Networking Basics



- Physical Layer
 - Example of Computing Effective Bandwidth:
 - Raw bandwidth: 10Mbps (mega-bits per second)
 - Sender overhead: 250 μ sec (0.00025 sec)
 - Receiver overhead: 300 μ sec (0.00030 sec)
 - Message size: 1000 bytes (8000 bits)
 - Distance: 1000km (Transmission speed = 150,000km/s)
 - What is the effective bandwidth?

$$\text{TransTime} = \frac{8000b}{10Mb/s} = \frac{8000b}{10b/\mu s} = 800\mu s$$

$$\text{TimeOfFlight} = \frac{1000km}{150,000km/s} = 6667\mu s$$

$$\text{TotalLatency} = 250 + 6667 + 800 + 300 = 8017\mu s \approx 0.008s$$

$$\text{EffBandwidth} = \frac{8000b}{0.008s} = 1Mb/s$$

- Note that this is only 1/10th of the raw bandwidth

Networking Basics



- Physical Layer
 - Maximum bandwidths of common media connection types:

Media Connection Type	Maximum Bandwidth (bps)
Serial cable	20K
USB 1&2	12M, 480M
ISDN	128K
DSL	1.5M down, 896K up
Cable	3M down, 256K up
LAN 10/100/1G BaseT	10M, 100M, 1G
Wireless 802.11 a/b/g	54M, 11M, 54M
Power line	14M
T1	1.5M

Note: Rates vary with direction

Typical Internet Latencies: 50-100ms

Source: Chapt 5.6 of Rabin

- Actual delivery is around 70% of maximum

Networking Basics



- Data Link Layer:
 - Puts data in frames and ensures error-free transmission
 - Controls the timing of the network transmission Adds frame type, address, and error control information
 - Examples of data link protocols are Ethernet for local area networks and PPP, HDLC and ADCCP for point-to-point connections
 - Network Interface Card (NIC): Performs these operations. Each NIC is associated with a MAC (media access control) address
 - All devices within a given subnetwork must have unique MAC addresses

Networking Basics



- Network Layer:
 - Performs end-to-end (source to dest) packet delivery, (whereas the data link layer is for node-to-node)
 - Performs network routing, flow control, data segmentation/de-segmentation, and error control
 - Famous example: The internet protocol (IP)
- IP Addresses:
 - IP version 4: (IPv4)
 - Address is 4 bytes, typically displayed in decimal: 255.8.128.16
 - Only $2^{32} = 4.3$ billion possible addresses
 - IP version 6: (IPv6)
 - Address is 16 bytes, displayed as 8 16-bit segments in hex: [2001:0db8:85a3:08d3:1319:8a2e:0370:7344]
 - Supports $2^{128} = 3.4 \times 10^{38}$ addresses. Roughly 1 address for every atom in everyone's body on the planet



廈門大學
XIAMEN UNIVERSITY

2.2 Networking and MMOGs

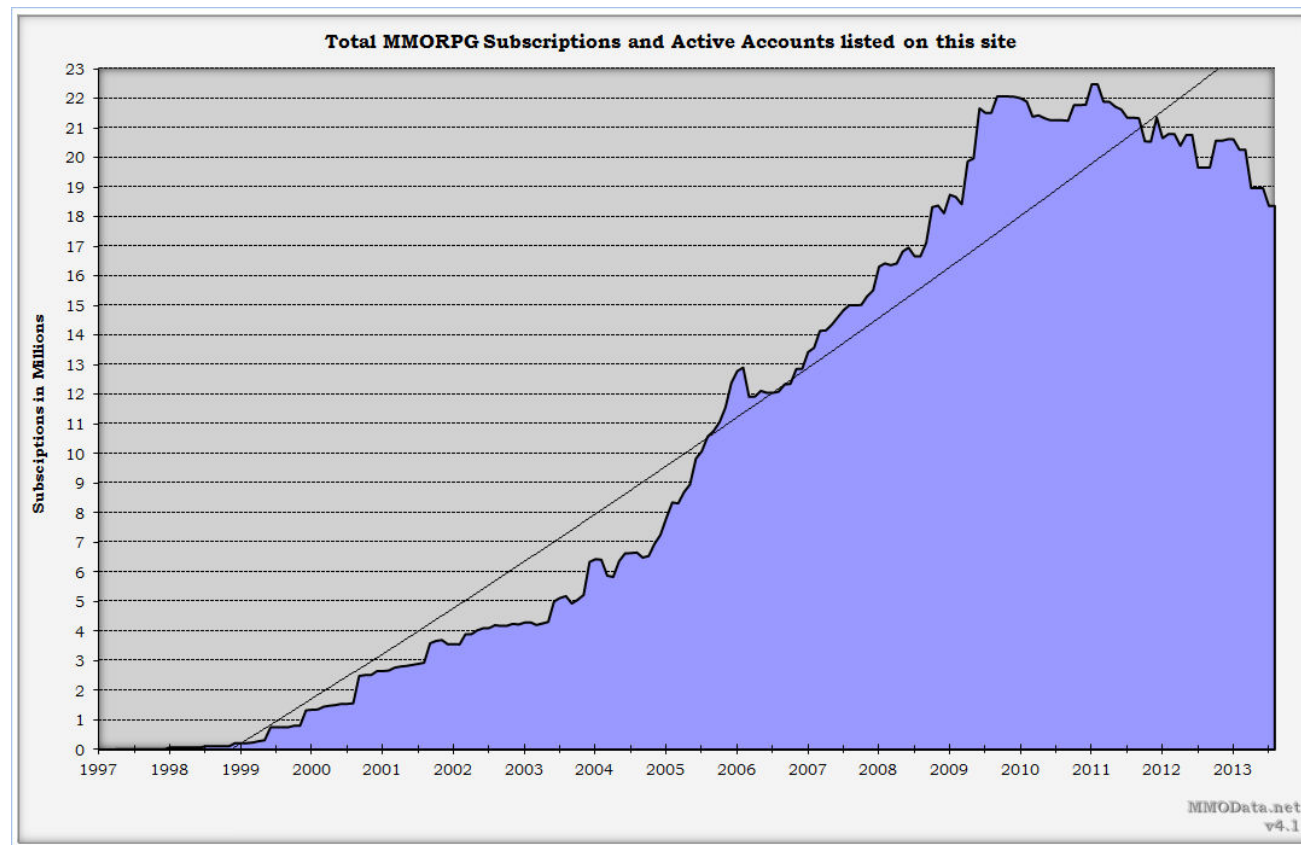
--自强不息，止于至善--

Games and Networking



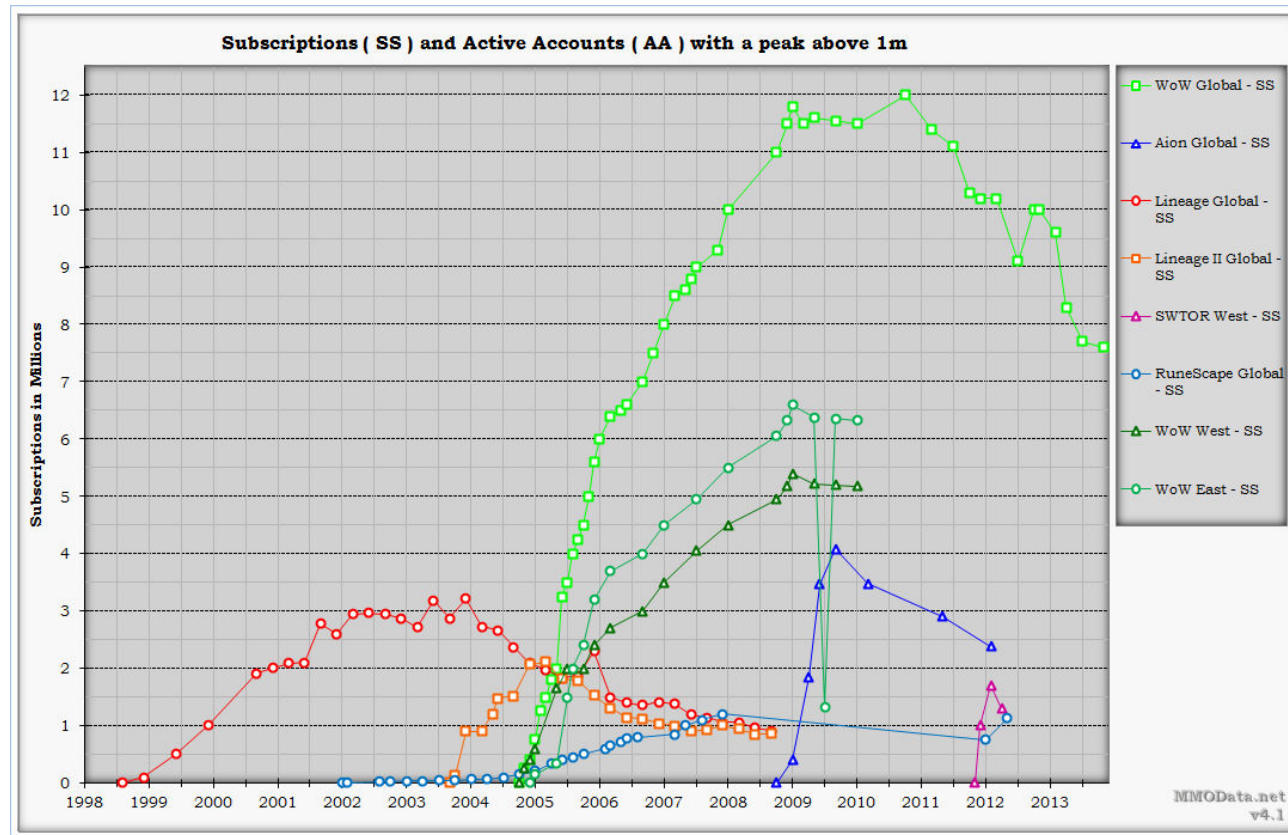
- Industry Information:
 - The top 17 game companies brought in \$33B in revenue in 2005
 - Major Players: Combined share \$23B
 - Nintendo
 - EA
 - Sony (games part)
 - Microsoft (games part)
- Online Games: About 50% of this is from online games:
 - Massively Multiplayer Online Games (MMOGs)
 - Smaller online play systems: Squad-on-squad play (under 32 players), or player-on-player games
 - Massively Multiplayer Online Role-Playing Games (MMORPGs) about \$4.3B

MMOG Subscription Growth



自强不息，止于至善

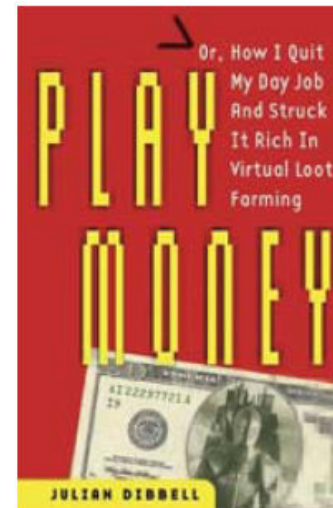
MMOG Subscription Growth



自强不息，止于至善

MMOG Rationale

- Why are MMOGs Popular?
- Humans are better at strategy than AI engines are:
 - Less predictable for general settings
- Social Interaction:
 - Peer group support
 - Natural language communication
 - Development of game “culture”
- Make money:
 - Professional gamers
 - Virtual economies
 - ...and the inevitable rise in “virtual crime”



Origins of MMOGs

- Quake (1996) :
 - First widely used 3D multiplayer online game
 - Difficult to find game servers:
 - Gamers exchanged IP addresses by email or gaming websites
 - No persistent state:
 - Short-lived ad-hoc fight-to-death sessions
- Advent of MMORPGs:
 - The Realm Online, Meridian 59, Ultima Online, Underlight and EverQuest in the late 1990s
- Scale:
 - 1995: 500 simultaneous players
 - 2000: Thousands
 - 2007: Hundreds of thousands



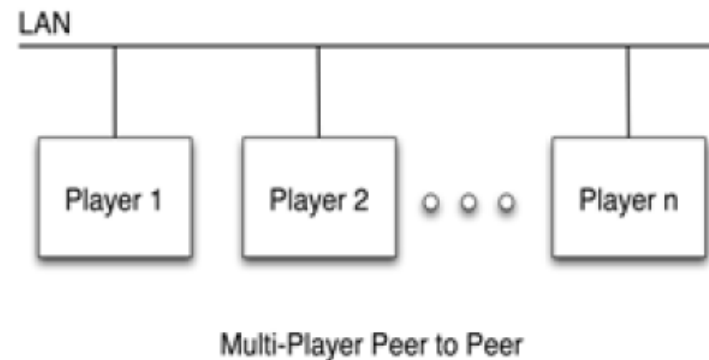
Persistent MMOGs



- Need to support millions of subscribers:
 - ...a few 100K concurrently
- Back-End Networking:
 - Authentication and billing
 - Ranking / black lists
 - Run-time mods/patches
 - Guard against denial of service attacks
- Front-End (in-game) Networking:
 - Network topology (client-server, peer-peer)
 - Persistence
 - Latency, bandwidth
 - Virtual economy (audits, gold farming, ...)
 - Distributed protocols

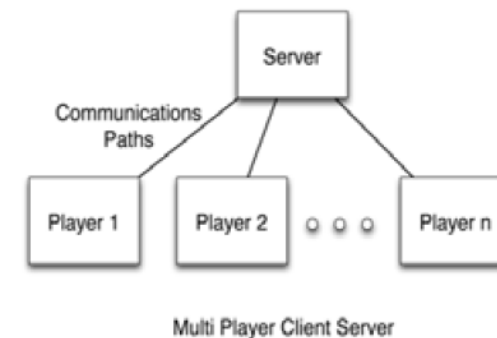
Online Game Architectures

- Peer-to-peer Architecture:
 - Each player communicates directly with all other players
- Possible complexity:
 - $O(n^2)$
 - Limited scalability
- Advantages:
 - Low latency, robust
- Disadvantages:
 - Demands high bandwidth, limited scalability
 - Persistence vs. redundancy tradeoff



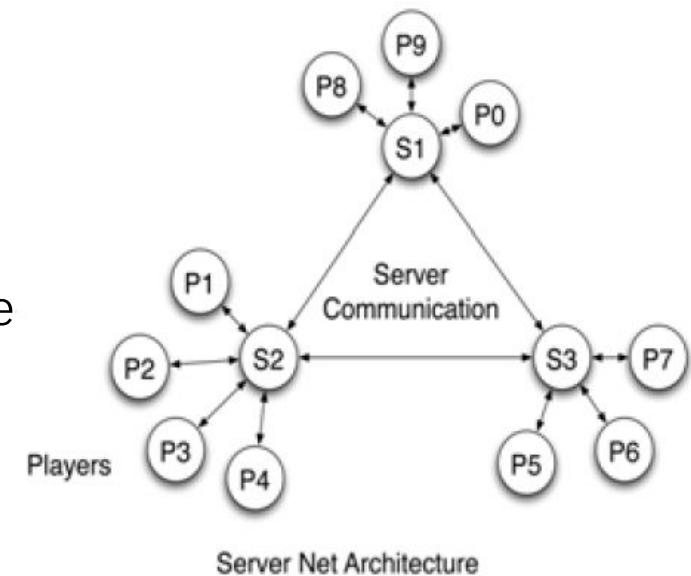
Online Game Architectures

- Client-Server Architecture:
 - Each player (client) communicates with server
- Advantages:
 - Scalable, usually requires less bandwidth.
 - Easier to provide persistence
- Disadvantages:
 - Server bottleneck: Higher latency
 - Server failure: Lower reliability
- Client Types:
 - Thin Client:
 - All simulation on server
 - Good for resource-poor clients: Cell phones, PDAs
 - Simulation Client:
 - Most simulation and world distributed amongst clients
 - Server maintains/updates state



Online Game Architectures

- Multiple Server Architecture:
 - Many distributed servers, each supporting a subset of the clients
- Advantages:
 - Reduced latency
 - Scalability to millions of players
 - Improved robustness
- Disadvantages:
 - Difficult to maintain consistency for persistence
- Possible Architectures:
 - Shards
 - Mirrors
 - Grids



Online Game Architectures



- Multi-Server Shards:
 - Each server simulates a different instantiation of the world
 - Each player plays within his own realm, and does not influence the other worlds
 - Minimal server-server traffic.
- Origin:
 - “...different images of the world, trapped in the shattered shards of a mystic gem...” Ultima Online story
- Examples:
 - Ultima Online
 - Neverwinter Nights
 - Silkroad Online
 - World of Warcraft (called Realms)



Online Game Architectures

- Multi-Server Mirrors:
 - Each server mirrors/replicates the same persistent world
- Advantages:
 - Common world view regardless of hosted server
- Disadvantages:
 - Heavy server-server traffic required for synchronization
- Examples:
 - Mankind
 - PlanetSide
 - A Tale in the Desert
 - Entropia Universe



Online Game Architectures

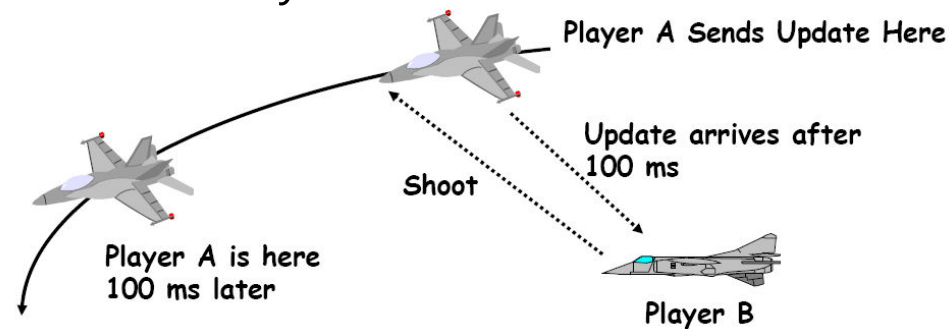


- Multi-Server Grid:
 - Each server (Sim) hosts a different region of the same persistent world
 - Sims communicate with each other on a grid structure
 - Users are transferred between servers as they move around
- Advantages:
 - Scalability for very large worlds
- Disadvantages:
 - Load balancing for high density regions
- Examples:
 - Second Life



Distributed Virtual Worlds

- Implementation is distributed, but appearance should be local
- Need to hide artifacts that suggest otherwise
- Each player sees real-time activity and reacts
- Actions should have immediate impact on the virtual world
- Major Challenge:
 - Latency: Delays induced by network structure



Distributed Virtual Worlds



- Latency:
 - All information received is out-of-date
 - Delays are variable and unpredictable
 - Maintaining the illusion of a consistent and persistent world is much harder when players interact
- Collisions:
 - Simple: Bullets, missiles (linear or parabolic)
 - Complex: Sword fighting, handshakes
 - Nightmare: Successive/concurrent collisions/responses

Distributed Virtual Worlds



- Visual latency:
 - 30Hz or higher (~ 30 ms)
- Response times: Depend on interaction speed
 - Real-Time Strategy: < 250 ms preferred, 250-500 ms playable, > 500 ms noticeable
 - First-Person Shooter: < 150 ms preferred
 - Car race games: < 100 ms preferred; 100 – 200 ms sluggish; > 500 ms out of control
- Predictability:
 - Predictable (even if sluggish) response considered better than variable (even if sometimes very fast)
- Haptic (touch) latency:
 - Preferred: Update rates > 1 KHz (< 1 ms) in force-feedback systems

Distributed Virtual Worlds

- Collision detection and response:
 - Did the collision occur?
 - If it did, when?
 - Velocities, forces, in play at that instant
- Environmental effects in a changing world:
 - Global illumination
 - Aural (sound) rendering
 - Both can be computationally intensive



Distributed Virtual Worlds



- Challenges: Managing Shared States
 - Shared repositories:
 - All servers maintain a common consistent description of the world
 - Absolute consistency!
 - Blind Broadcasts:
 - Owner of each state transmits its current state at regular intervals
 - Clients cache the most recent update information
 - No acknowledgements
 - Frequent state updates compensate for lost packets (hopefully)
 - Dead Reckoning:
 - Transmit state updates occasionally
 - Extrapolate from past updates to estimate the true shared state
 - Need to correct state when an update is received



廈門大學
XIAMEN UNIVERSITY

2.3

Socket Programming and Cheating

--自强不息，止于至善--

Sockets Programming



- Sockets:
 - Provide a means for programs (running perhaps on different machines) to communicate with one another
 - Types:
 - Stream sockets: (TCP: Transmission Control Protocol)
 - Reliable two-way connected communication streams
 - Items arrive in the order they were sent
 - Virtually error-free
 - Datagram sockets: (UDP: User Datagram Protocol)
 - No connection – Just generate a packet and send it
 - Packets may not arrive in the order they were sent
 - Packets may not arrive at all

Sockets Programming



- Sockets:
 - Provide a means for programs (running perhaps on different machines) to communicate with one another
 - Types:
 - Stream sockets: (TCP: Transmission Control Protocol)
 - Reliable two-way connected communication streams
 - Items arrive in the order they were sent
 - Virtually error-free
 - Datagram sockets: (UDP: User Datagram Protocol)
 - No connection – Just generate a packet and send it
 - Packets may not arrive in the order they were sent
 - Packets may not arrive at all

User Datagram Protocol (UDP)



- Most real-time games use the UDP protocol:
 - Faster and with lower overhead than TCP. Great!
- But there are consequences: UDP packets…
 - … are not guaranteed to arrive
 - … are not guaranteed to arrive in order
 - … are guaranteed to arrive with correct data, but have no protection from hackers intercepting and changing the data once it has arrived
 - … do not require a connection to be accepted. (Assists cheating. For example, intercept the packet "Give …blah… invulnerability," generate a copy of the message, and send it to the server anytime)
- …and global consequences:
 - Unlike TCP, UDP does not provide flow control or aggregation, and so it is possible to overrun the recipient's capacity

Socket APIs



- Low-level Socket APIs:
 - Berkeley Sockets API: for Unix
 - WinSock: for Windows
 - Both provide essentially the same functionality
- Basic capabilities: (for Berkeley sockets, but Winsock similar)
 - `socket( ... )`: Create a socket (of either type)
 - `gethostbyname( ... )`: Map hostname (e.g., "mysite.com") to IP address
 - `bind( ... )`: Connect a socket to a port on your local machine
 - `connect( ... )`: Connect to a remote machine and port
 - `listen( ... )`: Wait for input to arrive from a socket
 - `accept( ... )`: Accept a request from another host to connect to you
 - `send( ... )/recv( ... )`: Send and receive data
 - `sendTo( ... )/recvFrom( ... )`: Send/receive (for datagram sockets)
 - `close( ... )`: Terminate communication



RakNet: More Reliable UDP



- C++-based, open-source toolkit for (higher-level) UDP-based socket programming
- Supports client, server, and peer-to-peer communication
- Provides a layer over UDP, which addresses many of UDP's shortcomings
- Enhancements:
 - Can automatically resend lost packets
 - Can automatically order packets that arrived out of order
 - Protects transmitted data, and inform the programmer if that data was externally changed
 - Provides a connection layer that blocks unauthorized transmission
 - Transparently handles network issues such as flow control and aggregation (grouping many small transmissions into one packet)

RakNet: More Reliable UDP



- RakNet Standard Headers:

```
#include "MessageIdentifiers.h"  
#include "RakNetworkFactory.h"  
#include "RakPeerInterface.h"  
#include "RakNetTypes.h"
```

- RakPeerInterface:
- The main RakNet object

```
RakPeerInterface* peer =  
RakNetworkFactory::GetRakPeerInterface( );
```

- You will usually only generate one of these
- Base class for more specific objects: RakPeer, RakClient, and RakServer

RakNet: More Reliable UDP



- Connection as a Client:
 - Start up the network threads

```
peer->Startup( 1 30 &SocketDescriptor( ) 1 )
```

- 1: maximum number of connections. For a pure client, we use 1
- 30: thread sleep time (in msec)
 - 0 msec: good for games that need fast responses, such as FPS.
 - 30 msec: good response times with little CPU usage
- SocketDescriptor(): specifies the port/IP-address to listen on. Since we want a client, we don't need to specify anything
- – 1: Force RakNet to use a particular IP as host

RakNet: More Reliable UDP



- Connection as a Client: (cont.)

- Connect to the RakNet server

```
peer->Startup( maxConnectionsAllowed 30,  
               &SocketDescriptor( serverPort, 0 ), 1 );  
peer->SetMaximumIncomingConnections( maxPlayersPerServer );
```

- maxConnectionsAllowed: Maximum simultaneous connections
- 30: thread sleep time (in msec)
- SocketDescriptor: Which port to listen to
- maxPlayersPerServer: Maximum incoming connections to allow

RakNet: More Reliable UDP



- Read a Packet:

```
Packet* packet = peer->Receive( );
```

- Returns 0 (NULL) if no input
- Data may come from engine or other RakNet instances
- Packet Structure:

```
struct Packet {  
    SystemIndex systemIndex; // Server only: Sender index  
    SystemAddress systemAddr; // Who sent the packet  
    unsigned int length; // Data length in bytes (Deprecated)  
    unsigned int bitSize; // Data length in bits (Use this)  
    unsigned char* data ; // The data (Cast as needed)  
    bool deleteData; // Internal use  
};
```

RakNet: More Reliable UDP



- Send a Packet:

```
const char* message = "Hello World";  
peer->Send( (char*) message, strlen(message)+1,  
            HIGH_PRIORITY, RELIABLE, 0,  
            UNASSIGNED_SYSTEM_ADDRESS, true);
```

- message: data to be sent
- strlen(message)+1: length of data. Allow one additional byte for string's null ('\0') terminator
- HIGH_PRIORITY: Packet priority (also "LOW" and "MEDIUM")
- Reliability options:
 - UNRELIABLE: may not arrive and order of arrival arbitrary
 - UNRELIABLE_SEQUENCED: in order, but may not arrive
 - RELIABLE: guaranteed arrival, but order is not
 - RELIABLE_ORDERED: guaranteed arrival, and in order
 - RELIABLE_SEQUENCED: out of order packets are deleted
- Final arguments indicate a broadcast to all RakNet systems

RakNet: More Reliable UDP



- Shutting Down:

```
peer->Shutdown( 300 );  
...  
RakNetworkFactory::DestroyRakPeerInterface( peer );
```

- Shutdown: closes the connection. The connection can be restarted, using Startup()
 - 300: Indicates how long to wait (in msec) for remaining packets to be sent
- DestroyRakPeerInterface: Shut RakNet down and free all memory
- For more information:
 - See Raknet Manual and Tutorials. (Warning: Doxygen documentation appears to be out of date.)

Observations About Cheating



- Pritchard's Rules (Gamasutra article):
 - 1. If you build it, they will come—to hack and cheat
 - 2. Hacking attempts increase as a game becomes more successful
 - 3. Cheaters actively try to control knowledge of their cheats
 - 4. Your game, along with everything on the cheater's computer, is not secure — not memory, not files, not devices and networks
 - 5. Obscurity $\neq$ security
 - 6. Any communication over an open line is subject to interception, analysis and modification
 - 7. There is no such thing as a harmless cheat
 - 8 Trust in the server is everything in client-server games
 - 9. Honest players would like the game to tip them off to cheaters, hackers hate it

Why Care About Cheats?



- Achieving financial advantage:
 - Competitive games with prizes are the obvious example (casinos)
 - Virtual Economies:
 - People play the game, build good characters, and then auction them on eBay
 - If they can cheat to obtain better characters, they are achieving unfair financial advantage
- Ruining your game play => ruining your profits:
 - Online gaming is big business
 - Players tend to have a strong sense of fairness
 - If they believe they are being cheated they will stop playing and
- Cheating is principally an online issue:
 - Single player cheaters only affect themselves, so who cares?



廈門大學
XIAMEN UNIVERSITY

感谢您的观看
Thank you for watching